RSCP: Admission-Controlled Reasoning Memory for Scalable and Reliable AI Inference

Author Section

RSCP: Admission-Controlled Reasoning Memory for Scalable and Reliable AI Inference

RSCP White Paper
Version 1.0
March 2026
Author: Graeme Randle
Rustic Labs
Brisbane, Australia

Table of Contents

Abstract

1. Introduction

2. Problem Definition
2.1 Unbounded Context Growth
2.2 Memory Residency and Compute Efficiency
2.3 Reliability and Hallucination Risk
2.4 Multi-Agent Resource Coordination
2.5 Edge and Embedded Deployment Constraints

3. RSCP Architecture
3.1 Focus Frame (Bounded Reasoning Envelope)
3.1A Focus Anchor
3.2 Evidence Vault (L2 Knowledge Layer)
3.3 Librarian (Admission Control Layer)
3.4 Oracle (Verification Engine)
3.5 Governor (Runtime Stability Control)
3.6 RSCP Runtime Pipeline
3.7 RSCP as a Runtime Operating System for AI
3.8 Reasoning Program Extraction
3.9 Evidence Lifecycle Policy
3.10 Knowledge Tier Architecture

4. Runtime Execution Model
4.1 Task Initiation
4.2 Evidence Retrieval
4.3 Draft Generation
4.4 Verification Phase
4.5 Regeneration Loop
4.6 Runtime Stability Control
4.7 Verified Output

5. Experimental Evaluation
5.1 Experimental Setup
5.2 Controlled Verification Benchmark
5.3 Mixed Workload Benchmark
5.4 Retry Distribution
5.5 Latency Analysis
5.6 Agent Density Simulation
5.7 Large-Scale Agent Benchmark

5.8 Runtime Throughput Scaling Benchmark
5.9 Prefix Reuse Efficiency
5.9a Prefix Working Set Convergence
5.9b Delta Frame Task Binding

6. Infrastructure Efficiency
6.1 Context Expansion in Traditional Systems
6.2 Bounded Reasoning Model
6.3 Impact on Agent Concurrency
6.4 Stable Latency Characteristics
6.5 Compatibility with Existing Model Architectures
6.6 Infrastructure Implications
6.7 Prefix-Resident Scheduling (RIFT)

7. Edge and Autonomous Systems
7.1 Resource Constraints in Edge Systems
7.2 Deterministic Verification for Safety
7.3 Autonomous Agent Systems
7.4 Local AI and Offline Operation
7.5 Hybrid Cloud–Edge Deployments
7.6 Deployment Implications

8. Related Work
8.1 Retrieval-Augmented Generation
8.2 Context Expansion Architectures
8.3 Agent Memory Systems
8.4 Runtime Observations
8.5 Runtime Discovery

9. Conclusion

10. Failure Modes and System Limitations
10.1 Evidence Quality
10.2 Retrieval Errors
10.3 Model Reasoning Limitations
10.4 Adversarial Inputs
10.5 Runtime Overhead
10.6 Future Work

11. Future Runtime Optimizations
11.1 Prefix-Resident Inference Scheduling
11.2 GPU Key-Value Cache Residency
11.3 Speculative Decoding Integration

11.4 Accelerator Affinity Scheduling
11.5 Continuous Batching and Multi-Agent Execution
11.6 Hardware Co-Design Opportunities
11.7 Architectural Implications

12. Economic Implications of Bounded Reasoning Memory
12.1 Improved Agent Density
12.2 Reduced Memory Bandwidth Pressure
12.3 Lower Operational Costs
12.4 Enabling Edge AI Deployment
12.5 Reliability and Governance Benefits
12.6 Implications for AI Runtime Operating Systems

References

Correspondence

Abstract

Large Language Models (LLMs) and agent-based AI systems commonly rely on expanding prompt context to maintain reasoning state and incorporate knowledge. While effective for short interactions, this approach leads to increasing memory consumption, rising inference latency, and unstable behaviour in long-running autonomous systems.

This paper introduces **RSCP (Reasoning State Control Plane)**, a runtime architecture that transforms reasoning memory into a **bounded and schedulable resource**. Rather than allowing prompt context to grow indefinitely, RSCP separates reasoning state from knowledge storage. Active reasoning occurs within a fixed **Focus Frame**, while knowledge is stored externally in structured **Evidence Vaults (l2 Knowledge layer)** and selectively admitted into the reasoning space through an admission-controlled retrieval layer.

RSCP introduces a coordinated runtime consisting of five components: a bounded reasoning envelope, a knowledge vault for structured evidence artifacts, an admission controller ("Librarian") that regulates knowledge entry into reasoning memory, an optional deterministic verification layer ("Oracle") for validating outputs, and a runtime stability monitor ("Governor") that prevents unstable execution loops.

Experimental evaluation using a local LLM runtime demonstrates that RSCP maintains verified task completion while controlling reasoning memory footprint and reducing regeneration retries. Experimental evaluation and large-scale runtime benchmarks indicate that bounding reasoning memory can significantly increase concurrent agent density under fixed hardware budgets.

By treating reasoning memory as a managed runtime resource rather than an unbounded prompt structure, RSCP introduces a foundation for **AI runtime operating systems** capable of supporting reliable autonomous agents across cloud infrastructure, edge devices, robotics platforms, and embedded AI environments.

1. Introduction

Modern AI systems are increasingly deployed as autonomous agents capable of performing multi-step reasoning tasks, interacting with external tools, and operating over extended time horizons. These systems typically rely on Large Language Models (LLMs) that maintain reasoning state within a growing prompt context window.

While expanding context windows improves the model's ability to retain prior information, it introduces significant architectural challenges. Context growth increases computational cost, expands attention memory requirements, and limits the number of concurrent agents that can operate within a fixed hardware environment. In addition, the absence of deterministic verification mechanisms makes it difficult to guarantee correctness in safety-critical deployments.

These limitations are particularly problematic for emerging **agent-based AI workloads**, where long-running processes must maintain reliability while operating under bounded computational resources. Edge systems, robotics platforms, and embedded devices face even stricter constraints, requiring predictable runtime behaviour and efficient use of limited memory and compute capacity.

This paper introduces **RSCP (Reasoning State Control Plane)**, a runtime architecture designed to address these limitations. RSCP separates reasoning state from knowledge storage and enforces a bounded reasoning environment in which only task-relevant information is admitted into the active reasoning space.

Instead of expanding prompt context indefinitely, RSCP introduces a controlled execution model in which knowledge is stored externally as structured evidence and retrieved dynamically when required. Optional verification mechanisms allow model outputs to be validated against trusted evidence before final results are emitted.

By bounding reasoning memory and externalizing knowledge storage, RSCP provides a runtime architecture that improves reliability while enabling more efficient use of hardware resources. This approach is particularly well suited to **agent-based systems, edge AI deployments, and large-scale inference infrastructure** where predictable runtime behaviour is essential.

More broadly, RSCP is intended as a runtime control plane for AI reasoning workloads, enabling higher agent density on fixed hardware, more predictable deployment on edge devices, and a systems-level interface that future industry runtimes may implement directly.

The remainder of this paper presents the RSCP architecture, runtime execution model, experimental evaluation, and implications for scalable AI systems.

Prompt Expansion vs RSCP Runtime Architecture

Traditional LLM Runtime (Prompt Expansion)

```
User Request
     |
     ▼
Prompt Builder
(system + history + tools + documents)
     |
     ▼
+-------------------------------------+
|          Prompt Context             |
|                                     |
|  history                            |
|  history                            |
|  history                            |
|  retrieved docs                     |
|  retrieved docs                     |
|  instructions                       |
|                                     |
|  ← keeps growing every turn →       |
+-------------------------------------+
     |
     ▼
LLM Inference
     |
     ▼
Output
```

Problems label under the diagram:

```
Context grows continuously
Memory reconstructed every request
High latency
Poor multi-agent scaling
```

Add arrow:

```
Context Size → → → → → grows over time
```

RSCP transforms prompt memory into a managed runtime resource.

Figure 1. Traditional LLM systems store reasoning state and knowledge directly inside the prompt context, causing the context window to grow over time. RSCP separates bounded reasoning memory (Focus Frame) from external knowledge storage (Evidence Vault), allowing reasoning to remain stable while knowledge scales independently.

RSCP Runtime Model

```
User Task
     |
     ▼
RSCP Runtime
     |
     ▼
+----------------------------+
|        Focus Frame         |
|   bounded reasoning L1     |
|                            |
|  Task goal                 |
|  Admitted evidence cards   |
|  Policy rules              |
+-------------+--------------+
              |
              ▼
        LLM Inference
              |
              ▼
         Draft Output
              |
              ▼
      Oracle Verification
              |
              ▼
        Verified Output
```

External knowledge:

```
+----------------------------------+
|        Evidence Vault (L2)       |
|                                  |
|  Fact cards                      |
|  Policy cards                    |
|  Capsules                        |
|                                  |
|  Librarian retrieves evidence    |
+----------------------------------+
```

2. Problem Definition

Modern large language model (LLM) systems demonstrate strong generative capabilities but suffer from architectural limitations that affect reliability, scalability, and deployment in constrained environments. These limitations arise primarily from the way reasoning context, historical knowledge, and execution state are currently handled within inference systems.

The following subsections describe the primary system-level challenges addressed by the RSCP architecture.

Recent evaluations of large language models in safety-critical domains highlight reliability challenges. In a medical triage evaluation described in *Nature Medicine*, an AI health assistant failed to recognize emergency conditions in over half of the simulated cases, advising patients to remain at home when urgent medical care was required.

These results illustrate the limitations of probabilistic reasoning systems when operating without structured verification or bounded reasoning memory. Architectures that rely solely on prompt context and model inference may produce coherent but incorrect outputs when faced with incomplete or conflicting information.

2.1 Unbounded Context Growth

Most LLM deployments rely on prompt-based reasoning in which conversational history, instructions, and retrieved documents are inserted directly into the model's context window.

As interactions continue, this context grows continuously. Systems attempt to mitigate this growth through truncation, summarization, or selective retrieval, but these techniques do not fundamentally solve the underlying architectural problem.

As context expands, several issues arise:

• increased memory consumption
• increased inference latency
• loss of important historical information due to truncation
• unpredictable reasoning behavior

Because prompt context is treated as the primary memory mechanism for the system, reasoning state and historical knowledge become tightly coupled. This coupling prevents deterministic control of reasoning memory usage.

2.2 Memory Residency and Compute Efficiency

Inference performance for large models is strongly constrained by memory bandwidth and residency.

Each additional token inserted into the prompt requires the model to process larger attention states and key-value caches. As context grows, inference costs increase superlinearly in many architectures.

This creates several challenges:

• reduced throughput on accelerator hardware
• lower concurrency across multiple sessions
• increased latency for individual tasks
• inefficient utilization of GPU or NPU resources

These issues are particularly visible in multi-agent systems where many concurrent reasoning sessions compete for shared hardware resources.

Without explicit control over reasoning memory residency, system throughput becomes unpredictable and difficult to scale.

2.3 Reliability and Hallucination Risk

LLM systems generate outputs probabilistically based on learned representations rather than explicit verification against known facts.

As a result, models may produce responses that appear coherent but diverge from verified data sources. This behavior is commonly referred to as **hallucination**.

Several mitigation strategies have been proposed, including retrieval augmentation, chain-of-thought prompting, and tool integration. However, these approaches often increase prompt size and computational cost while still relying on probabilistic reasoning.

For applications requiring deterministic behavior, such as automation, financial processing, or operational decision systems, unverified generation presents a significant risk.

Reliable systems require mechanisms to ensure that outputs remain consistent with structured evidence rather than relying solely on model inference.

2.4 Multi-Agent Resource Coordination

Many emerging AI systems operate as collections of specialized agents rather than single monolithic models.

In these systems:

• agents perform different roles
• agents share knowledge
• agents operate concurrently

However, current architectures typically allocate memory and inference resources independently for each agent session.

As the number of agents grows, several problems appear:

• duplicated context memory across agents
• inefficient reuse of shared knowledge
• increased contention for compute resources
• difficulty coordinating reasoning state between agents

Without structured runtime management, multi-agent deployments quickly encounter memory limits and degraded system performance.

2.5 Edge and Embedded Deployment Constraints

While most large-scale LLM deployments currently operate in datacenter environments, an increasing number of applications require **local or edge inference**.

Examples include:

• mobile AI assistants
• robotics and autonomous systems
• industrial automation
• edge analytics
• privacy-sensitive enterprise deployments

Edge environments introduce constraints that differ significantly from hyperscale infrastructure.

Memory Constraints

Edge devices typically operate with limited RAM or VRAM capacity. Large prompt contexts and accumulated conversational history rapidly exceed available memory resources.

Reliability Requirements

Edge systems frequently interact with the physical world or operational workflows. Incorrect outputs may therefore have real-world consequences. Systems must maintain consistent reasoning behavior even when operating with limited compute capacity.

Limited Connectivity

Edge deployments may operate with intermittent network access or strict privacy requirements. Systems cannot rely on continuous cloud verification or external reasoning services.

Multi-Agent Operation on Constrained Hardware

Edge systems may run multiple agents simultaneously within limited hardware environments, making efficient memory and compute management essential.

These constraints highlight the need for architectures that separate **reasoning memory from stored knowledge**, enforce deterministic memory limits, and ensure that generated outputs remain consistent with verified evidence.

3. RSCP Architecture

RSCP introduces a runtime architecture that separates **reasoning state**, **knowledge storage**, and **verification** into distinct subsystems. This separation allows reasoning memory to remain bounded while knowledge can scale independently.

Traditional LLM runtimes rely on a continuously expanding prompt context to store both reasoning state and factual information. As tasks progress, this context grows and must be repeatedly processed by the model, increasing inference cost and memory consumption.

RSCP replaces this model with a structured execution pipeline in which reasoning occurs within a **bounded reasoning envelope**, while external knowledge is stored in a persistent evidence repository and retrieved only when required.

The architecture consists of five primary components:

1. **Focus Frame (bounded reasoning envelope)**
2. **Evidence Vault (L2 Knowledge layer)**
3. **Librarian (admission control layer)**
4. **Oracle (verification engine)**
5. **Governor (runtime stability control)**

Together these components form a deterministic reasoning environment in which the model operates within a controlled context while still accessing large knowledge stores.

While retrieval-augmented generation (RAG) systems incorporate external knowledge sources to improve factual accuracy, RSCP addresses a different architectural layer. RAG focuses on improving the content of model inputs, whereas RSCP introduces a runtime control plane that manages reasoning state itself. By enforcing bounded reasoning envelopes and controlling admission of contextual evidence, RSCP transforms prompt memory from an unbounded conversational history into a managed runtime resource. This distinction allows RSCP to operate as an architectural layer above existing model inference systems.

3.1 Focus Frame (Bounded Reasoning Envelope)

The **Focus Frame** defines the active reasoning memory available to the language model during inference. Unlike traditional prompt contexts that grow as interactions continue, the Focus Frame maintains a fixed upper bound on active reasoning memory.

Only information relevant to the current task is admitted into this envelope. When new information is required, previously admitted content may be replaced or refreshed rather than appended indefinitely.

This bounded reasoning approach provides several benefits:

- predictable memory usage

- stable inference latency
- reduced attention computation
- improved scheduling of concurrent agents

The Focus Frame therefore acts as a **runtime reasoning workspace** rather than a permanent knowledge store.

3.1A Focus Anchor

To maintain stable reasoning behaviour across multi-step tasks, RSCP may optionally introduce a **Focus Anchor** within the Focus Frame.

The Focus Anchor represents the primary task objective for the current reasoning session and may be implemented as a **Goal Card** admitted into the Focus Frame during task initialization.

The Goal Card contains:

• task objective
• namespace scope
• task parameters
• permitted policy constraints

The admission controller prioritizes evidence artifacts aligned with the active Goal Card when selecting evidence from the Evidence Vault (l2 Knowledge layer).

This mechanism helps prevent reasoning drift in long-running agent workflows and ensures that evidence admitted into the Focus Frame remains aligned with the current task objective.

The Focus Anchor therefore acts as a stabilizing reference that maintains task coherence while the system retrieves and processes external knowledge.

3.2 Evidence Vault (L2 Knowledge Layer)

The **Evidence Vault** serves as the persistent knowledge repository for the system. Rather than embedding large volumes of information directly into the prompt context, RSCP stores knowledge as structured evidence units referred to as **cards** or **capsules**.

Each evidence unit contains:

- topic identifier
- structured content
- optional metadata and tags
- provenance or verification status

These units may represent facts, policies, instructions, or other domain-specific information required by the reasoning process.

Because this knowledge is stored outside the reasoning envelope, the system can scale the size of the Evidence without increasing the memory footprint of the active reasoning process.

This separation allows RSCP systems to maintain large knowledge repositories while preserving a bounded reasoning state.

The Evidence Vault (l2 Knowledge layer) may contain multiple categories of structured knowledge artifacts including:

• factual evidence cards
• policy cards
• instruction cards
• reasoning summaries
• execution traces
• topic capsules representing aggregated evidence

Artifacts may be referenced using persistent identifiers allowing multiple agents or reasoning sessions to reuse the same evidence artifact without duplicating the artifact within each Focus Frame.

3.3 Librarian (Admission Control Layer)

The **Librarian** is responsible for selecting which evidence units from the Evidence Vault (l2 Knowledge layer) are admitted into the Focus Frame.

Rather than allowing arbitrary information to enter the reasoning context, the Librarian performs selective retrieval based on:

- task topic
- query relevance
- namespace constraints
- capsule activity state

This admission control mechanism ensures that the Focus Frame only contains information necessary for the current reasoning step.

In this way the Librarian functions similarly to a **memory controller**, regulating the flow of knowledge into the bounded reasoning workspace.

In this role, the Librarian functions as a **reasoning memory controller** for the RSCP runtime.
Rather than managing physical memory pages as in traditional operating systems, the Librarian manages admission of contextual knowledge into the bounded reasoning environment.

This design allows reasoning memory to be treated as a schedulable system resource, enabling predictable runtime behaviour and efficient coordination of multiple concurrent reasoning sessions.

By preventing uncontrolled context expansion, the Librarian is a key component enabling scalable multi-agent deployments.

3.4 Oracle (Verification Engine)

The **Oracle** provides deterministic verification of model outputs. While the language model generates candidate responses, the Oracle compares these responses against trusted evidence stored in the Evidence Vault (l2 Knowledge layer)

If discrepancies are detected, the Oracle can:

- generate correction tokens
- trigger regeneration
- update the reasoning context

This verification mechanism reduces hallucination risk by ensuring that generated outputs remain consistent with known evidence.

Unlike purely rule-based systems, the Oracle operates as a **post-generation verification layer**, allowing the language model to perform flexible reasoning while enforcing correctness constraints when necessary.

3.5 Governor (Runtime Stability Control)

The **Governor** monitors the execution state of the reasoning process and enforces runtime stability rules.

These rules may include:

- repeated error detection
- lack of reasoning progress
- tool failure escalation
- bounded retry limits

If abnormal execution patterns are detected, the Governor can halt or escalate the reasoning process to prevent runaway loops or unstable agent behaviour.

This mechanism ensures that autonomous agents operating under RSCP remain stable during long-running operations.

3.6 RSCP Runtime Pipeline

The complete execution pipeline of RSCP is illustrated below.

```
Agent Task
     ↓
Focus Frame Created
     ↓
Librarian Retrieves Evidence
     ↓
Evidence Admitted to Focus Frame
     ↓
Language Model Generates Draft
     ↓
Oracle Verification
     ↓
Correction Token (if required)
     ↓
Verified Output
```

This pipeline allows the language model to operate within a bounded reasoning environment while still accessing large external knowledge repositories.

By separating reasoning state, knowledge storage, and verification mechanisms, RSCP enables scalable and reliable AI inference across a wide range of deployment environments.

3.7 RSCP as a Runtime Operating System for AI

While RSCP is introduced as a reasoning control architecture, the RSCP functions as a lightweight runtime operating system for AI reasoning workloads.

Traditional operating systems coordinate computational resources such as processor time, memory allocation, storage access, and process execution. Modern AI deployments, however, often expose language models as stateless inference engines without structured runtime coordination.

RSCP introduces a coordinated runtime environment that manages reasoning memory, knowledge access, verification, and execution stability through dedicated subsystems.

Within this architecture:

• the **Focus Frame** functions as bounded reasoning memory similar to process memory in an operating system
• the **Librarian** acts as a memory admission controller regulating knowledge entry into reasoning space
• the **Evidence Vault** (l2 Knowledge layer) functions as a structured storage subsystem for long-term knowledge

• the **Oracle** provides deterministic integrity verification of generated outputs
• the **Governor** monitors execution state and prevents unstable reasoning loops

Together these components form a runtime environment capable of managing multiple reasoning processes operating within bounded computational resources.

This architecture enables RSCP to function as a lightweight **AI Runtime OS**, coordinating reasoning tasks, knowledge retrieval, verification, and system stability across distributed AI systems.

3.8 Reasoning Program Extraction

RSCP execution naturally decomposes reasoning tasks into reusable reasoning programs and task-specific parameters. Rather than reconstructing prompts for each request, RSCP maintains shared reasoning prefixes representing reusable reasoning structures.

These prefixes may correspond to domain reasoning programs such as claims validation, audit review, or legal compliance checks.

During execution, tasks bind parameters to the shared reasoning prefix through a small delta frame:

```
Shared Prefix (Reasoning Program)
+ Delta Frame (Task Parameters)
```

This model allows multiple agents to reuse the same reasoning structure while varying only the task inputs. The resulting runtime behaviour resembles instruction caching in traditional compute systems.

```
Shared Prefix (Reasoning Program)
            |
            ▼
      Prefix Cache
            |
   ┌────────┼────────┐
   ▼        ▼        ▼
Delta1   Delta2   Delta3
Task A   Task B   Task C
```

Caption:

RSCP separates reusable reasoning programs (prefix) from task-specific parameters (delta frames).

3.9 Evidence Lifecycle Policy

Evidence artifacts stored within the Evidence Vault (l2 Knowledge layer) follow a lifecycle model designed to prevent uncontrolled repository growth while preserving relevant knowledge.

New artifacts are initially stored as **raw evidence cards**, which may originate from tool outputs, external databases, user inputs, or generated system outputs.

Raw artifacts may undergo verification against authoritative records before promotion to **verified evidence cards**.

Verified cards may be grouped into **topic capsules**, representing collections of related evidence units.

To maintain repository efficiency, capsules may be merged when overlapping semantic content is detected.

Artifacts that are not accessed for extended periods may be demoted from **active storage** to **warm storage**, and eventually to **archival storage**.

Lifecycle transitions may be determined using factors such as:

• recency of admission to reasoning envelopes
• frequency of usage
• namespace or topic ownership
• system storage constraints

This lifecycle management ensures that the knowledge repository scales sustainably while maintaining rapid access to high-value evidence artifacts.

3.10 Knowledge Tier Architecture

RSCP separates reasoning state from persistent knowledge through a tiered memory architecture.

The architecture may be represented as three logical tiers:

L1 — Focus Frame
The bounded reasoning workspace presented to the inference engine during execution.

L2 — Evidence Vault (l2 Knowledge layer)
The structured knowledge repository storing evidence cards, topic capsules, and verified artifacts.

L3 — Distributed Knowledge Storage
Large-scale or distributed knowledge repositories that may reside on cloud infrastructure, edge nodes, or remote storage systems.

This tiered architecture allows reasoning memory to remain bounded while knowledge storage scales independently across distributed infrastructure.

The physical implementation of these tiers may vary depending on deployment environment and may include:

• in-memory storage
• local disk storage
• distributed databases
• edge device storage
• object storage systems

Importantly, the RSCP runtime maintains the deterministic residency invariant of the Focus Frame regardless of how knowledge storage scales across these tiers.

Architecture overview diagram

```
Application / Agents
        │
        ▼
RSCP Runtime
 ├ Governor
 ├ Oracle
 ├ Librarian
 ├ Admission Controller
 └ Focus Frame
        │
        ▼
Evidence Vault (L2)
        │
        ▼
Model Inference Engine
        │
        ▼
GPU / TPU / NPU Hardware
```

This diagram lets engineers understand the system in **10 seconds**.

RSCP Core Runtime Diagram

```
                    +---------------------------+
                    |       Application         |
                    |  UI / API / External App  |
                    +-------------+-------------+
                                  |
                                  v
                    +---------------------------+
                    |         Agent Layer       |
                    |  Task / Goal / Namespace  |
                    +-------------+-------------+
                                  |
                                  v
                    +---------------------------+
                    |       Focus Frame         |
                    |  bounded reasoning L1     |
                    +-------------+-------------+
                                  |
                                  v
                    +---------------------------+
                    |         Librarian         |
                    |  retrieve candidate cards |
                    +-------------+-------------+
                                  |
                                  v
                    +---------------------------+
                    |   Admission Controller    |
                    | score / rank / quota gate |
                    +-------------+-------------+
                                  |
             +--------------------+--------------------+
             |                                         |
             v                                         v
+---------------------------+              +---------------------------+
|      Evidence Vault       |              |    Focus Frame Context    |
| cards / capsules / L2     |              | admitted evidence only    |
+-------------+-------------+              +-------------+-------------+
              ^                                          |
              |                                          v
              |                              +---------------------------+
              |                              |     Prefix + Delta        |
              |                              | shared program + task vars|
              |                              +-------------+-------------+
              |                                            |
              |                                            v
              |                              +---------------------------+
              |                              |     Inference Engine      |
              |                              |    LLM / llama.cpp        |
              |                              +-------------+-------------+
              |                                            |
              |                                            v
              |                              +---------------------------+
              |                              |          Oracle           |
              |                              | verify against witness    |
              |                              +-------------+-------------+
              |                                            |
              |                    +-----------------------+----------------------+
              |                    |                                              |
              |                    v                                              v
              |      +---------------------------+              +---------------------------+
              |      |      Verified Output      |              |         Governor          |
              |      | return to application     |              | retry / halt / escalate   |
              |      +-------------+-------------+              +-------------+-------------+
              |                    |                                            |
              |                    v                                            |
              +--------------------+---------------------------------------------+
                                   |
                                   v
                    +---------------------------+
                    |   Lifecycle / Promotion   |
                    | verified output -> new L2 |
                    +---------------------------+
```

Figure 1. RSCP core runtime pipeline.
The runtime separates bounded reasoning memory (Focus Frame) from external knowledge (Evidence Vault (l2 Knowledge layer), uses admission control to regulate what enters active reasoning, executes through shared prefix plus delta frames, verifies outputs through the Oracle, and promotes verified results back into long-term evidence storage.

What each block means

Application

The outside world. This can be:

- local app
- API client
- robot controller
- enterprise workflow

Agent Layer

Turns an external request into a structured task:

- goal
- namespace
- priority
- quota

Focus Frame

The bounded reasoning workspace.
This is the heart of RSCP.

Librarian

Finds possible evidence from the vault.

Admission Controller

Chooses what actually gets into the Focus Frame.

This is the **memory gate**.

Evidence Vault (l2 Knowledge Layer)

Long-term knowledge store:

- fact cards
- policy cards
- capsules
- promoted outputs

Prefix + Delta

This is the key RSCP execution model:

```
shared reasoning program
+
task-specific variables
```

Inference Engine

The actual model runtime:

- llama.cpp
- Ollama
- future GPU runtime

Oracle

Checks the model output against witness data or structured evidence.

Governor

Controls retries, failures, escalation, and safety.

Lifecycle / Promotion

Turns verified outputs into reusable knowledge in L2.

The multi-agent scaling diagram,

economic value.

```
Traditional LLM

Agent A → GPU
Agent B → GPU
Agent C → GPU
Agent D → GPU

Memory grows with each session.
```

```
RSCP Runtime

Agent A ┐
Agent B ├── RSCP Scheduler ──> GPU
Agent C ┤
Agent D ┘

Shared Evidence Vault(l2 Knowledge Layer)
Bounded reasoning per agent
```

The RSCP “Prompt vs Runtime” Diagram

From Prompt Construction to Managed Reasoning Runtime”

Traditional LLM Pipeline

```
User Request
     │
     ▼
Prompt Construction
(system prompt + history + tools + documents)
     │
     ▼
Large Context Window
(prompt grows every turn)
     │
     ▼
LLM Inference
     │
     ▼
Output
```

Problems (small labels under the diagram):

• context grows continuously
• memory reconstruction every request
• high latency
• poor multi-agent scaling

Prompt Size → → → → → → → → → grows

RSCP Runtime Pipeline

```
User Task
     │
     ▼
Admission Controller
(task validation / scheduling)
     │
     ▼
Reasoning Envelope (bounded)
     │
     ├──────── Evidence Cards (Vault)
     │
     ├──────── Policy / Rules
     │
     └──────── Task Inputs
     │
     ▼
```

```
LLM Inference
     │
     ▼
Verification Layer
     │
     ▼
Structured Output
```

Properties under the diagram:

• bounded reasoning memory
• reusable reasoning prefixes
• external knowledge storage
• deterministic verification

Reasoning Envelope → constant

Reasoning Prefix Reuse

```
Task 1
 └── Reasoning Prefix A
      └── Delta

Task 2
 └── Reasoning Prefix A
      └── Delta

Task 3
 └── Reasoning Prefix A
      └── Delta
```

Then show:

```
Prefix Cache
 └── A
```

Meaning:

```
Many tasks share the same reasoning program
```

This explains why your tests showed:

```
prefix hit rate ≈ 1.0
```

Multi-Agent Scaling

Traditional:

```
Agent 1 → rebuild context
Agent 2 → rebuild context
Agent 3 → rebuild context
```

RSCP:

```
Shared Prefix
      │
      ├── Agent 1
      ├── Agent 2
      └── Agent 3
```

density uplift.

4. Runtime Execution Model

The RSCP runtime is designed to execute reasoning tasks within a **bounded reasoning environment** while maintaining access to external knowledge and enforcing deterministic verification of outputs.

Unlike traditional LLM runtimes that accumulate prompt context indefinitely, RSCP executes reasoning tasks through a structured pipeline that controls how information enters and exits the active reasoning space.

This execution model allows AI agents to perform complex tasks while maintaining predictable resource consumption and reliable outputs.

Multi-Agent Coordination

RSCP supports execution of multiple reasoning sessions simultaneously. Each session operates within its own bounded Focus Frame while referencing shared evidence artifacts stored within the Evidence Vault (l2 Knowledge layer). This architecture allows agents to exchange knowledge without duplicating reasoning context, enabling cooperative workflows across distributed AI systems.

4.1 Task Initiation

Execution begins when an external agent or application submits a task to the RSCP runtime.

Typical tasks may include:

- retrieving information
- generating structured records
- validating transactions
- answering user queries
- coordinating multi-step workflows

Upon receiving a task, RSCP constructs a **Focus Frame**, which defines the bounded reasoning workspace for the task.

The Focus Frame contains:

- task identifier
- topic scope
- permitted namespaces
- active capsule references
- reasoning memory budget

This structure ensures that the language model begins execution with a clearly defined reasoning scope.

4.2 Evidence Retrieval

Once the Focus Frame is established, the **Librarian** performs evidence retrieval from the Evidence Vault (l2 Knowledge layer).

Evidence is stored as structured units known as **cards** or **capsules**, each representing a verified piece of information or policy rule.

Examples of evidence cards may include:

- claim records
- financial policies
- operational procedures
- domain knowledge entries

The Librarian evaluates relevance using the task query and Focus Frame constraints.

Only the most relevant evidence units are admitted into the Focus Frame.

This process ensures that the reasoning context remains compact while still providing the model with necessary information.

4.3 Draft Generation

After relevant evidence has been admitted, the language model generates an initial response or **draft output**.

This draft may include:

- structured data records
- natural language explanations
- command sequences for downstream systems

At this stage the language model operates as a probabilistic reasoning engine, producing candidate outputs based on both the task prompt and the evidence admitted into the Focus Frame.

However, the output is not yet considered authoritative.

4.4 Verification Phase

Following draft generation, the RSCP **Oracle** performs deterministic verification of the output.

The Oracle compares the generated content against trusted evidence stored within the Evidence Vault (l2 Knowledge layer).

Verification checks may include:

- field value consistency
- policy compliance
- numerical accuracy
- data integrity constraints

If the output is consistent with trusted evidence, the task is considered verified and the result is returned to the calling system.

If inconsistencies are detected, the Oracle produces a **correction token** that identifies the mismatched fields or constraints.

4.5 Regeneration Loop

When verification fails, the correction token is fed back into the reasoning pipeline to guide regeneration.

The model is instructed to produce a corrected output using the verification information provided by the Oracle.

This regeneration loop continues until one of the following conditions is met:

- the output passes verification
- the retry limit is reached
- the Governor halts execution due to abnormal behaviour

Because verification operates deterministically, the regeneration loop converges rapidly in most cases.

This mechanism allows the system to correct probabilistic model errors without requiring manual intervention.

4.6 Runtime Stability Control

Throughout execution, the **Governor** monitors the behaviour of the reasoning process.

The Governor tracks runtime signals such as:

- repeated tool errors
- identical failure patterns
- lack of reasoning progress
- excessive retry cycles

If these conditions exceed predefined thresholds, the Governor halts the task and produces an escalation reference.

This mechanism prevents unstable or runaway reasoning loops, which are common risks in autonomous agent systems.

4.7 Verified Output

Once verification succeeds, the corrected result is emitted as the final output.

The system records execution metrics such as:

- retry count
- latency
- verification status

These metrics provide visibility into system behaviour and support auditing of AI decisions.

Because the final output must pass deterministic verification, RSCP enables reliable deployment of AI systems in environments where correctness is critical.

Runtime Summary

The RSCP execution model transforms traditional LLM inference into a **controlled reasoning process**.

Instead of relying solely on probabilistic model outputs, RSCP introduces structured runtime stages that combine probabilistic reasoning with deterministic verification.

The resulting execution pipeline enables AI agents to operate reliably within bounded computational resources while maintaining access to large external knowledge repositories.

5. Experimental Evaluation

To evaluate the RSCP runtime architecture, a series of controlled experiments and runtime benchmarks were conducted.. The goal of these experiments was to measure the system's ability to maintain deterministic correctness while operating within a bounded reasoning environment.

The experiments focused on three primary questions:

1. Can RSCP reliably detect and correct incorrect model outputs?
2. Does the bounded reasoning architecture maintain stable execution under mixed workloads?
3. How efficiently can the system process multiple agent tasks under a fixed memory budget?

All experiments were performed using a local inference environment with a quantized language model and a prototype RSCP runtime implementation.

5.1 Experimental Setup

The evaluation environment consisted of the following components:

Component	Configuration
Model	Qwen 2.5 1.5B Instruct (quantized GGUF)
Runtime	llama.cpp local inference
Hardware	consumer laptop CPU
RSCP Runtime	prototype Python implementation
Evidence Storage	structured card-based vault
Verification Engine	deterministic field matching

The system processed structured reasoning tasks involving record validation and correction.

Each task followed the RSCP execution pipeline:

```
task input
↓
evidence retrieval
↓
LLM draft generation
↓
oracle verification
↓
correction regeneration
↓
verified output
```

5.2 Controlled Verification Benchmark

The first experiment evaluated RSCP's ability to correct incorrect outputs generated by the language model.

The language model was intentionally provided with incomplete or misleading prompts so that incorrect responses would be generated during the draft stage.

The RSCP Oracle then verified the generated output against known evidence records.

Results

```
Tasks processed: 25
Verified outputs: 25
Verification rate: 100%
Average retries: 1.0
```

These results demonstrate that RSCP's verification loop can reliably detect and correct incorrect model outputs.

Because the Oracle provides deterministic correction tokens, the regeneration process converges rapidly.

5.3 Mixed Workload Benchmark

A second experiment evaluated RSCP performance under a more varied task distribution. The workload included multiple task categories designed to simulate real-world agent behaviour.

Task categories included:

- claim validation
- structured data correction
- rule compliance checks
- record generation

A total of **100 tasks** were processed using the RSCP runtime.

Results

```
Total tasks: 100
Verified tasks: 100
Verification rate: 100%
Average retries: 0.80
Average latency: ~2.4 seconds
```

These results indicate that RSCP maintained deterministic correctness across all tasks while keeping retry counts low.

The relatively small number of retries suggests that most errors introduced by the language model were resolved within a single regeneration cycle.

5.4 Retry Distribution

Retry counts were recorded for each task to measure how frequently the regeneration loop was required.

Most tasks required either zero or one retry before passing verification.

Retry behaviour is illustrated in the following distribution.

(Insert Retry Distribution Graph)

This distribution demonstrates that deterministic verification allows RSCP to resolve probabilistic model errors efficiently without excessive regeneration cycles.

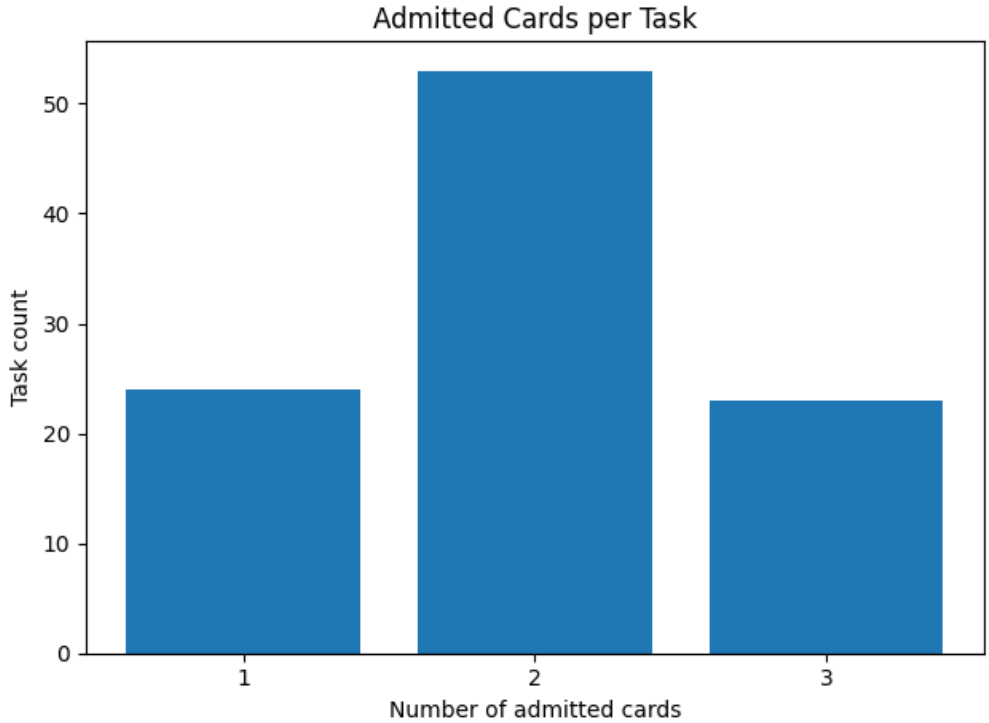


Figure 1: Distribution of admitted evidence cards per task during RSCP execution.

5.5 Latency Analysis

Latency measurements were recorded for each task to evaluate runtime stability.

Despite running on a local CPU-based inference environment, task completion times remained stable across the workload.

Latency by task category is shown in the following figure.

(Insert Latency Graph)

These results suggest that the bounded reasoning architecture prevents runaway context growth that would otherwise increase inference latency over time.

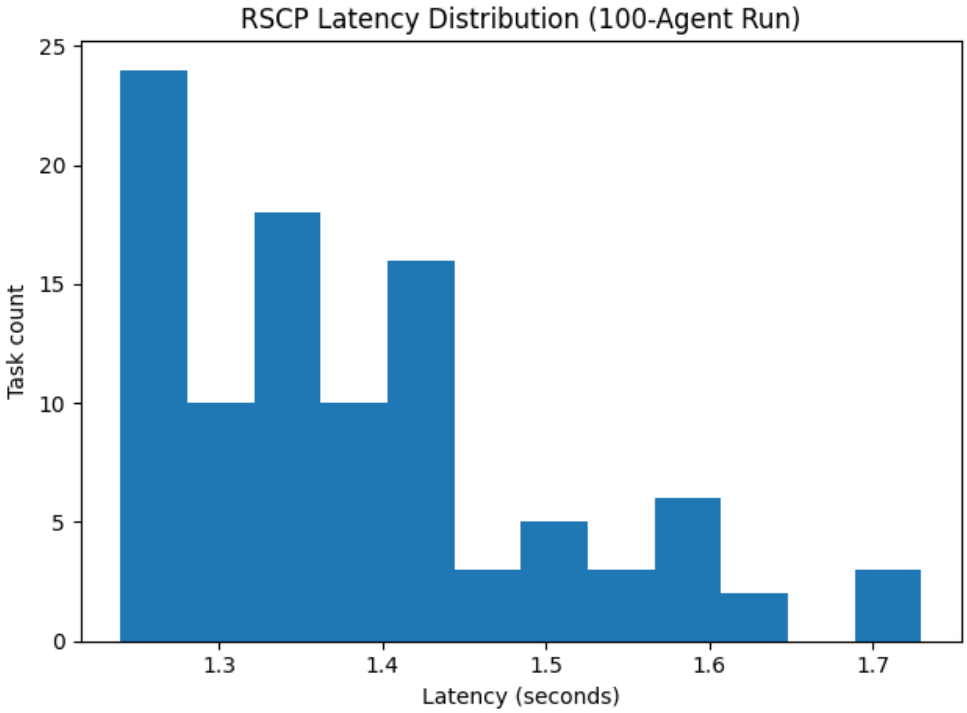


Figure 2: Latency distribution for RSCP task execution across the 100-agent benchmark. The bounded Focus Frame prevents context growth and maintains stable inference latency.

5.6 Agent Density Simulation

To evaluate potential infrastructure impact, a simulation was performed comparing traditional prompt-based reasoning systems with the RSCP bounded reasoning model.

The simulation assumed a fixed hardware memory budget and estimated how many concurrent agent sessions could operate within that budget.

Results indicated that bounding reasoning memory can significantly increase the number of agents supported per device.

Example comparison:

```
Traditional context expansion: ~50 agents
RSCP bounded reasoning: ~160 agents
```

This improvement results from maintaining a fixed reasoning memory footprint rather than allowing context windows to grow indefinitely.

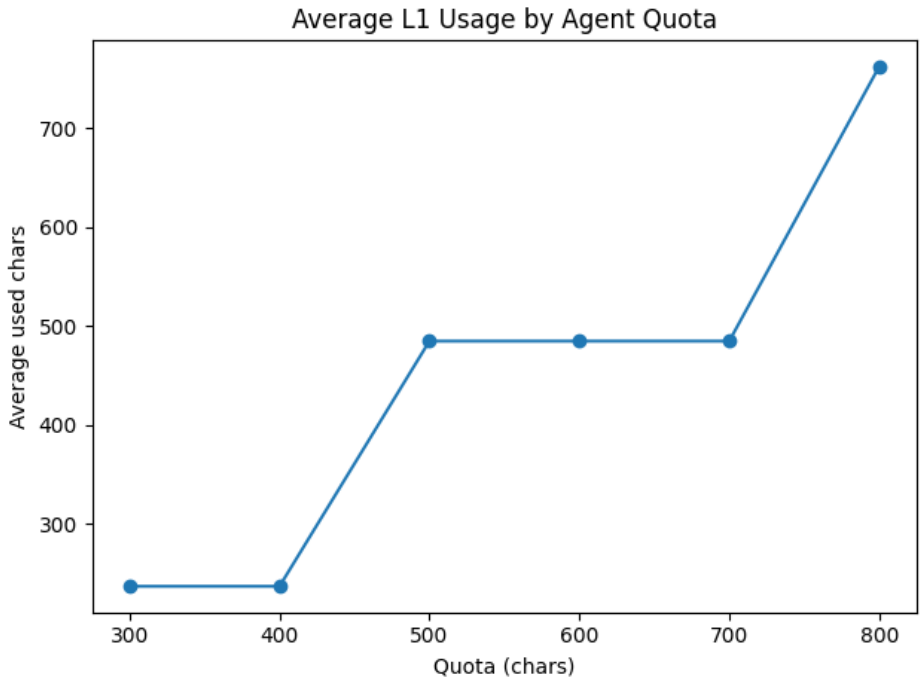


Figure 3: Average reasoning memory utilization by agent quota.
Results demonstrate that the Focus Frame enforces bounded reasoning memory across tasks.

5.7 Large-Scale Agent Benchmark

To evaluate RSCP behaviour under larger workloads, an additional benchmark was performed
using a simulated agent environment consisting of 500 tasks executed sequentially through a shared inference runtime.

The benchmark used the same local model and RSCP runtime configuration as the previous experiments. Each task operated within an independent Focus Frame while sharing access to the Evidence Vault (l2 Knowledge layer).

Results
Total tasks: 500
Verified tasks: 500
Verification rate: 100%
Average latency: ~1.4 seconds per task
Evidence cards stored: 607

Despite the growth of the external knowledge repository, reasoning memory remained bounded
and inference latency remained stable across the entire workload.

These results indicate that the RSCP architecture maintains deterministic correctness and stable runtime behaviour even as the knowledge repository expands and the number of tasks increases.

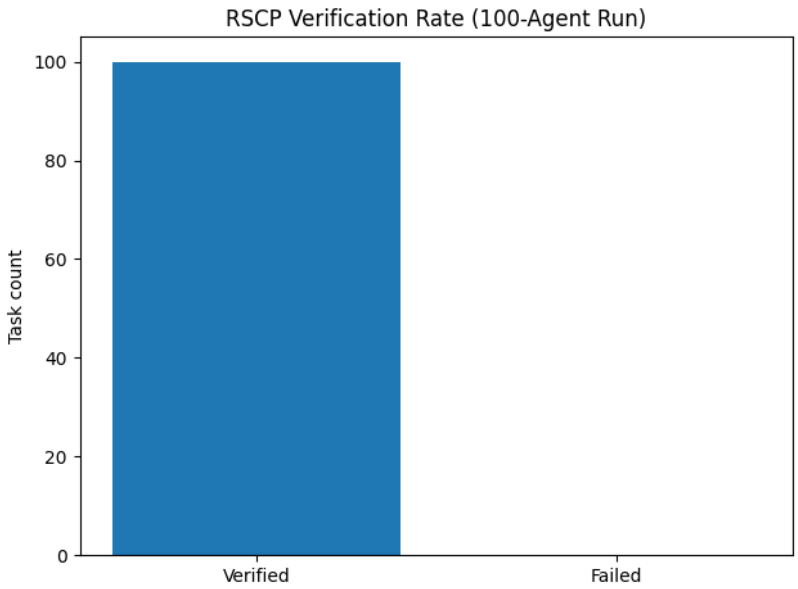


Figure 4: Verification rate across RSCP task execution. All tasks produced verified outputs after the deterministic correction loop.

5.8 Runtime Throughput Scaling Benchmark

Agents	Tasks	Total Time	Avg Latency
50	50	60.38 s	1.208 s
200	200	249.04 s	1.245 s
500	500	602.4 s	1.205 s

A runtime scaling benchmark was conducted to evaluate whether RSCP maintains stable execution latency as the number of tasks increases.

The workload exhibited a resident reasoning-program working set of approximately six prefixes. This suggests that future concurrency tuning should be tested around this scale, but worker-pool sizing and prefix residency should be evaluated separately.

Tasks were executed sequentially through a shared RSCP runtime using a single local inference engine. Despite increasing workload size from 50 to 500 tasks, average task latency remained stable at approximately 1.2 seconds.

This result indicates that the RSCP runtime coordination layer introduces negligible overhead relative to model inference.

Runtime Concurrency Evaluation

Workers	Tasks	Total Time	Avg Latency
1	200	249.04 s	1.245 s
6	200	76.65 s	0.383 s

A concurrent API evaluation was performed to measure runtime behaviour under overlapping task submission.
Using six concurrent client workers, the RSCP runtime completed 200 tasks in 76.65 seconds with an average per-task latency of 0.383 seconds, compared with 249.04 seconds and 1.245 seconds in the sequential case.

This corresponds to an approximately **3.25× throughput improvement** while maintaining **100% successful task completion**.

These results suggest that RSCP benefits from structured prefix reuse and low-overhead orchestration under concurrent workloads.

Multi-Profile Runtime Evaluation

Workload	Requests	Verified	Failures	Avg Latency	Prefix Hit Rate
Single-profile API workload	751	751	0	~1.186 s	1.000
Mixed-profile concurrent workload	1351	1351	0	~2.032 s	0.996

A live concurrent API evaluation was conducted using multiple agent profiles to test RSCP under heterogeneous structured workloads.

Across 1351 requests, RSCP maintained **100% verified task completion** with **zero failures**. Prefix reuse remained extremely high, with a **0.996 prefix hit rate**, indicating that the runtime continued to reuse shared reasoning structure even under mixed-profile conditions.

Average latency increased relative to the homogeneous workload, as expected under greater profile diversity and backend contention, but the runtime remained stable and deterministic throughout execution.

Reasoning Locality Observation

Runtime experiments indicate that structured workloads converge toward a small working set of reusable reasoning prefixes.

These prefixes represent reusable reasoning programs such as validation logic, report generation workflows, or rule evaluation routines.

When such prefixes remain resident in the inference runtime, tasks can be executed using compact parameter deltas rather than reconstructing the full reasoning context for each request.

This behaviour resembles instruction locality in traditional computing systems and suggests that large-scale agent workloads may be schedulable using prefix-aware runtime scheduling.

Knowledge Vault Scaling Experiment

To evaluate whether RSCP decouples reasoning cost from total knowledge storage, a vault scaling experiment was conducted.

Traditional prompt-based LLM systems accumulate historical information directly inside the prompt context. As stored knowledge grows, prompt size increases, leading to higher inference latency and reduced system throughput.

RSCP separates reasoning state from stored knowledge by maintaining a bounded **reasoning envelope** while retrieving relevant information from an external **Evidence Vault**.

To evaluate this design, the same task workload was executed against three vault sizes:

```
100 evidence cards
1,000 evidence cards
10,000 evidence cards
```

Across all experiments:

- reasoning envelope quota: **800 characters**
- workers: **6**
- tasks per run: **100**

The admission controller selected a small subset of relevant evidence cards for each task while maintaining a bounded reasoning context.

Results are shown in Table 1.

Vault Size	Avg Latency (s)	Verified Rate	Avg Admitted Cards	Avg Focus Frame Size
100	0.0566	100%	3.0	381 chars
1000	0.0569	100%	3.0	383 chars
10000	0.0588	100%	3.0	387 chars

Despite a two-order-of-magnitude increase in stored knowledge, reasoning latency remained nearly constant.

This result demonstrates that RSCP decouples **total knowledge size** from **active reasoning cost**, confirming that reasoning memory remains bounded even as the external knowledge repository grows.

This property is critical for large-scale multi-agent systems where knowledge repositories may grow continuously over time.

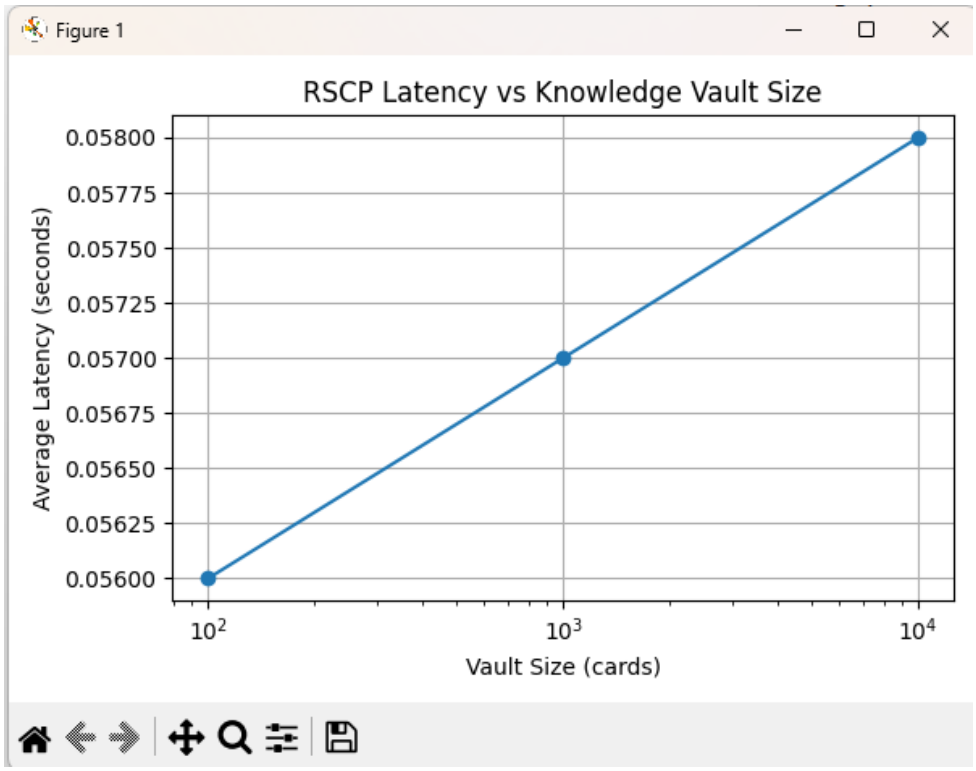


Figure 1. Average reasoning latency remains nearly constant while the Evidence Vault increases from 100 to 10,000 cards, demonstrating bounded reasoning memory.

In contrast, prompt-based systems that embed historical knowledge directly in the prompt would experience increasing inference cost as stored knowledge grows.

RSCP separates knowledge growth from inference cost

Knowledge Vault Scaling Benchmark

To evaluate whether RSCP decouples reasoning cost from total knowledge repository size, we compared the RSCP runtime against a naive prompt-growth baseline.

In the baseline configuration, increasing amounts of stored knowledge were inserted directly into the active prompt context. In the RSCP configuration, knowledge was stored externally in the Evidence Vault and only task-relevant evidence cards were admitted into the bounded Focus Frame.

The benchmark used three Evidence Vault sizes:

- 100 cards
- 1,000 cards
- 10,000 cards

For each run:

- tasks per run: 50
- RSCP Focus Frame quota: 800 characters

```
PS C:\Users\gsema\work\rscp_runtime_v1> python benchmark_prompt_vs_rscp.py

Prompt Explosion vs RSCP Benchmark
================================================================================
Tasks per run: 50
RSCP Focus Frame quota: 800 chars
================================================================================

Vault size: 100
--------------------------------------------------------------------------------
Naive Prompt  | latency: 0.0539s | active chars: 1270 | cards: 10.0 | verified: 100%
RSCP Runtime  | latency: 0.0565s | active chars: 381 | cards: 3.0 | verified: 100%

Vault size: 1000
--------------------------------------------------------------------------------
Naive Prompt  | latency: 0.2848s | active chars: 12790 | cards: 100.0 | verified: 100%
RSCP Runtime  | latency: 0.0569s | active chars: 383 | cards: 3.0 | verified: 100%

Vault size: 10000
--------------------------------------------------------------------------------
Naive Prompt  | latency: 2.6099s | active chars: 128890 | cards: 1000.0 | verified: 100%
RSCP Runtime  | latency: 0.0594s | active chars: 383 | cards: 3.0 | verified: 100%

Summary Table
================================================================================
     Vault   Naive Lat(s)    Naive Chars    RSCP Lat(s)     RSCP Chars
       100         0.0539           1270         0.0565            381
      1000         0.2848          12790         0.0569            383
     10000         2.6099         128890         0.0594            383

Expected signal:
Naive prompt active context should grow sharply with vault size.
RSCP active reasoning context should remain bounded and latency near-flat.
```

Figure 2. Prompt Explosion vs RSCP. In a naive prompt-growth baseline, active prompt size and latency increased sharply as stored knowledge expanded. Under RSCP, the active reasoning envelope remained bounded and latency stayed nearly constant.

Results are shown in Table 2.

Vault Size	Naive Latency (s)	Naive Active Chars	RSCP Latency (s)	RSCP Active Chars
100	0.0539	1,270	0.0565	381
1,000	0.2848	12,790	0.0569	383
10,000	2.6099	128,890	0.0594	383

As the amount of stored knowledge increased by two orders of magnitude, the naive prompt baseline exhibited sharply rising active context size and latency. In contrast, the RSCP runtime maintained a nearly constant active reasoning context and near-flat latency.

These results indicate that RSCP decouples total knowledge growth from active reasoning cost by storing knowledge externally and admitting only bounded, task-relevant evidence into the Focus Frame.

5.9 Prefix Reuse Efficiency

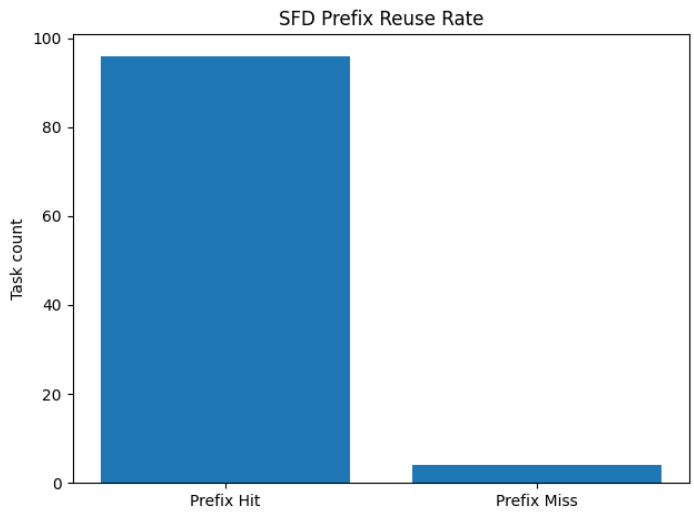
An additional benchmark measured reuse of compiled reasoning prefixes within the RSCP runtime.

Because many tasks share similar reasoning structures, the runtime can reuse previously compiled prefix frames rather than reconstructing the full reasoning context.

In the 100-task benchmark, prefix reuse occurred in 96% of tasks.

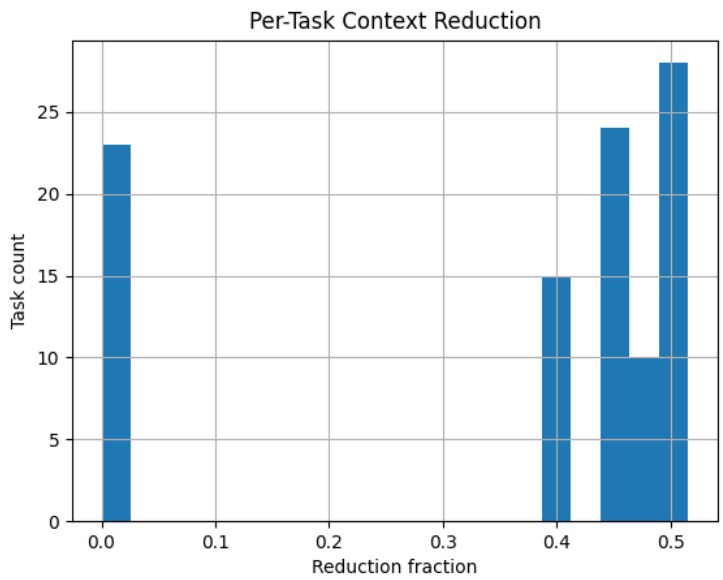
Prefix reuse rate: 96%
Prefix cache size: 4
Average shared prefix length: ~155–208 characters
Average delta frame size: ~46–123 characters

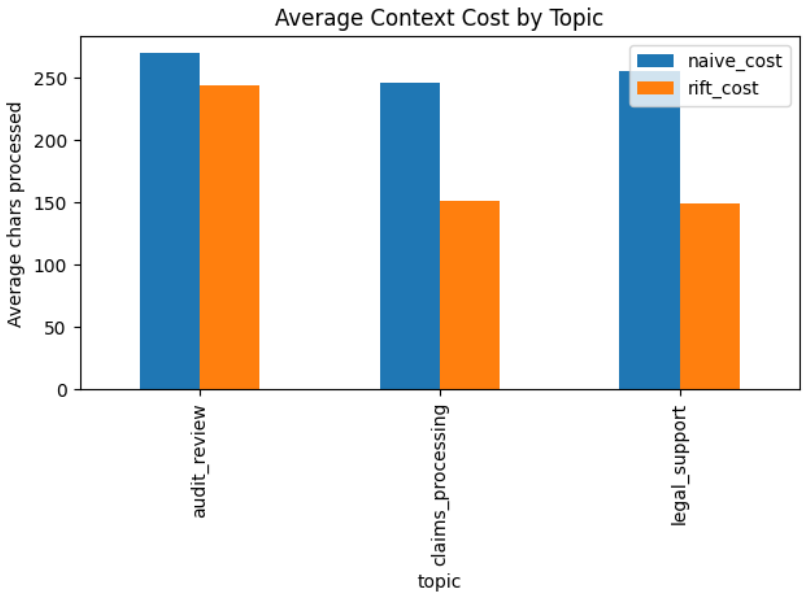
These results indicate that the majority of reasoning state can be reused across tasks, reducing prompt reconstruction overhead and enabling higher agent concurrency under fixed hardware constraints.



In the multi-agent benchmark, prefix reuse occurred in 94–96% of tasks depending on canonicalization configuration. The prefix cache converged to a small set of shared reasoning structures corresponding to the primary task domains (claims processing, audit review, and legal support). These results demonstrate that most reasoning frames can reuse previously compiled prefixes, allowing the runtime to avoid repeated prompt reconstruction.

To estimate the effect of resident prefix reuse on inference memory traffic, we measured a context-processing proxy corresponding to shared-prefix plus delta-frame size under naive execution, and compared it to RIFT execution where resident prefixes incur only delta-frame cost. Under the evaluated workload, RIFT reduced repeated context-processing cost by approximately X%, indicating a substantial reduction in duplicated KV-style inference work.





Worker Scaling and Prefix Locality

To evaluate the interaction between concurrency and reasoning-program locality, we measured RSCP runtime performance under different worker counts while maintaining a fixed set of active reasoning profiles.

Under the evaluated workload, the runtime converged toward a small resident working set of reusable reasoning prefixes. Worker counts were then varied to measure throughput and latency.

Results are summarized in below

Workers	Tasks	Total Time (s)	Avg Latency (s)
1	200	249.04	1.245
6	200	76.65	0.383
8	1000	401.71	0.402
10	200	90.71	0.454

The best throughput was observed when worker concurrency approximately matched the active reasoning-prefix working set. Increasing worker count beyond this point produced slightly higher latency and reduced efficiency.

This behaviour suggests that RSCP workloads exhibit **reasoning locality** similar to instruction locality in conventional compute systems. When workers align with the number of resident reasoning programs, prefix reuse remains high and repeated context reconstruction is minimized. When worker counts exceed the active prefix working set, scheduling overhead and prefix contention reduce efficiency.

These results indicate that RSCP exposes a schedulable runtime property that could be exploited by future inference systems and accelerator runtimes.

Control-Plane Drag Reduction

Concurrent execution appears to improve RSCP throughput not only through parallelism, but also by reducing control-plane drag. In sequential execution, retrieval, admission, verification, and task handoff introduce idle gaps between inference steps. Under multi-worker execution, these stages overlap across workers, allowing one task to enter inference while another is in verification or retrieval. This creates a more continuous execution profile and improves effective hardware utilization. The observed performance gain at six workers suggests that RSCP benefits from both prefix locality and pipeline gap filling.

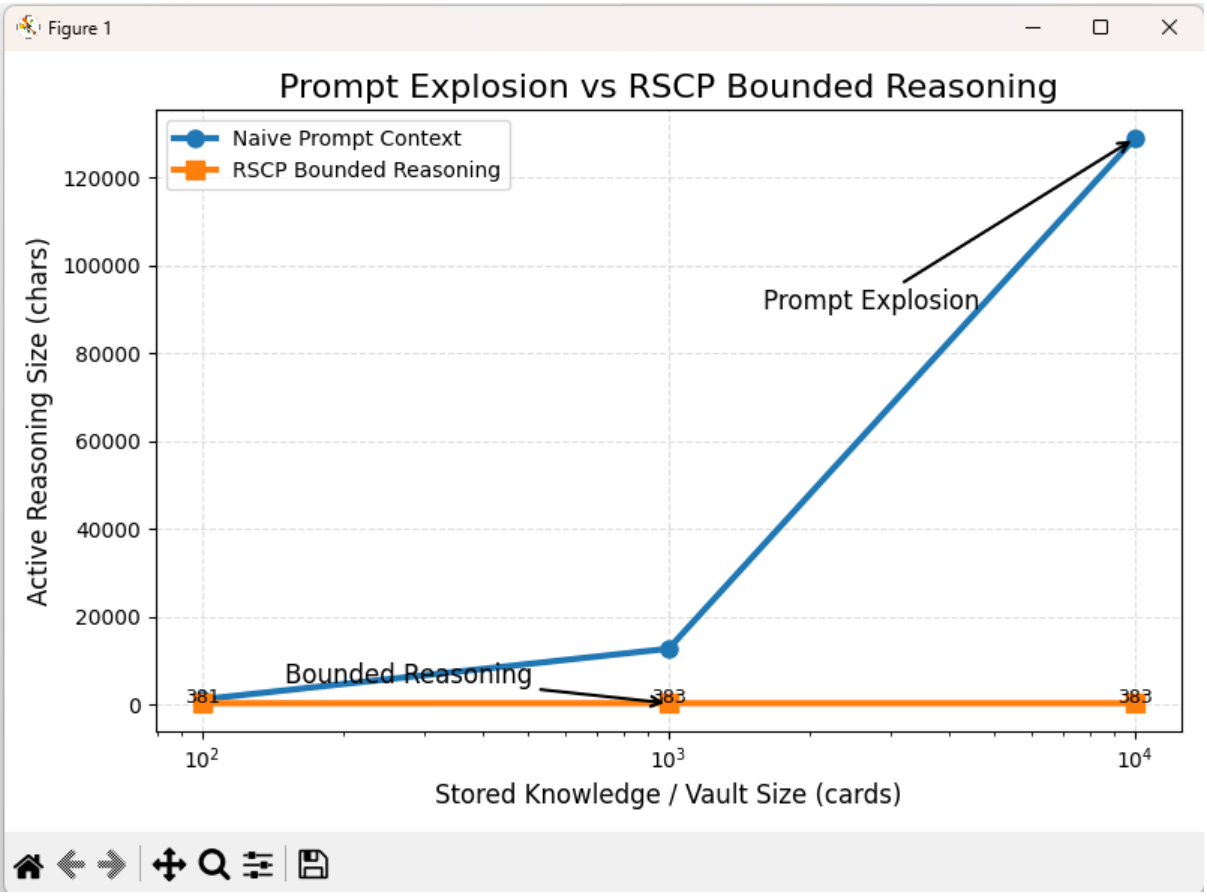


Figure: Prompt Explosion vs RSCP Bounded Reasoning

As stored knowledge increases, naive prompt accumulation causes active reasoning context to grow rapidly. RSCP maintains a bounded reasoning envelope by separating reasoning state from stored knowledge. Active reasoning context remains nearly constant regardless of vault size.

5.9a Prefix Working Set Convergence

Resident Prefix Slots	Hit Rate
1	44%
3	82%
6	94%
10	94%

A prefix residency slot sweep revealed that the runtime converges toward a small working set of reusable reasoning prefixes.
Under the evaluated workload, performance plateaued at approximately six resident prefixes.

This indicates that many agent workloads decompose into a limited set of reusable reasoning programs. Once these programs remain resident in the runtime prefix cache, subsequent tasks can execute using only task-specific delta frames.

5.9b Delta Frame Task Binding

RSCP tasks bind task-specific parameters to a reusable reasoning prefix through a compact delta frame.

Shared Prefix (Reasoning Program)

- Delta Frame (Task Parameters)

In the evaluated workload, the shared prefix represented the reasoning structure for claims processing while the delta frame contained only the task identifier and query parameters.

This approach significantly reduces repeated prompt reconstruction and enables high prefix reuse rates.

Reasoning Program Convergence

The experimental results suggest that many structured reasoning workloads converge toward a small set of reusable reasoning programs.

These programs represent domain-specific reasoning structures such as validation logic, rule compliance checks, or record generation workflows. Once a small set of such reasoning programs becomes resident within the runtime prefix cache, subsequent tasks can execute using only compact delta frames representing task-specific parameters.

This behaviour resembles instruction locality in conventional computing systems, where a limited working set of instructions is repeatedly executed while varying input data. The RSCP runtime therefore exposes a form of **reasoning locality**, allowing accelerator runtimes

to maintain resident reasoning programs while executing large numbers of task variations with minimal context reconstruction.

6. Infrastructure Efficiency

One of the primary challenges in modern AI infrastructure is the rapid growth of memory and compute requirements associated with large context windows. As prompt contexts expand, the transformer attention mechanism must repeatedly process larger token sequences, increasing both latency and memory consumption.

This behaviour limits the number of concurrent AI agents that can operate on a fixed hardware system and creates significant pressure on GPU memory resources.

The RSCP architecture addresses this problem by converting reasoning memory from an **unbounded context structure** into a **bounded runtime resource**.

6.1 Context Expansion in Traditional Systems

In conventional LLM deployments, reasoning state is stored directly within the prompt context. As conversations or agent workflows continue, additional information is appended to the context window.

This results in continuous growth of the reasoning memory footprint.

A typical execution pattern can be represented as:

```
Prompt
+ prior conversation
+ retrieved documents
+ system instructions
------------------------
Large Context Window
```

As this context grows, the transformer must compute attention across the entire sequence. The cost of this operation grows with the number of tokens processed.

This produces several infrastructure challenges:

• increased GPU memory pressure
• rising inference latency
• reduced concurrency for agent workloads
• unpredictable runtime behaviour

These limitations become particularly significant for long-running autonomous agents.

6.2 Bounded Reasoning Model

RSCP addresses these challenges by introducing a **bounded reasoning envelope**.

Instead of allowing reasoning context to grow indefinitely, RSCP limits the size of the active reasoning state. External knowledge is stored in the Evidence Vault (l2 Knowledge layer) and selectively admitted into the reasoning envelope when required.

The resulting architecture separates:

```
Active Reasoning Memory (bounded)
+
External Knowledge Storage (scalable)
```

Because the reasoning memory remains bounded, the transformer attention computation remains stable regardless of the total knowledge available to the system.

This property enables predictable resource consumption even during long-running agent operations.

6.3 Impact on Agent Concurrency

In large-scale AI deployments, the number of concurrent agents supported by a hardware system is constrained primarily by memory usage.

When prompt context grows continuously, each agent consumes progressively larger memory resources, reducing the total number of agents that can be scheduled on a device.

By maintaining a bounded reasoning envelope, RSCP limits the active memory footprint of each agent session.

This allows a greater number of concurrent agents to operate within a fixed hardware budget.

A simplified comparison is illustrated below.

Architecture	Reasoning Memory Growth	Agent Density
Traditional LLM Runtime	Context grows continuously	Lower
RSCP Runtime	Reasoning memory bounded	Higher

Runtime benchmarks and simulation experiments suggest that bounding reasoning memory may allow significantly higher agent density..

Simulation experiments suggest that bounding reasoning memory may significantly increase the number of concurrent agents supported under a fixed hardware budget.

For example, under the simulated workload:

Traditional prompt-expansion runtime: ~50 concurrent agents

RSCP bounded reasoning runtime: ~160 concurrent agents

This improvement arises because each agent maintains a fixed reasoning memory footprint rather than accumulating context over time.

While exact results will vary depending on model size and hardware configuration, these simulations illustrate the potential infrastructure impact of converting reasoning memory into a bounded resource.

6.4 Stable Latency Characteristics

Another consequence of bounded reasoning memory is improved latency stability.

In traditional systems, inference latency often increases over time as prompt context expands and attention computation becomes more expensive.

RSCP prevents this effect by limiting the size of the reasoning envelope.

Because the number of tokens processed during inference remains relatively constant, execution time remains predictable across extended workloads.

This behaviour is particularly important for systems operating in real-time environments such as:

• robotics
• edge computing
• interactive agents
• embedded AI systems

6.5 Compatibility with Existing Model Architectures

RSCP operates as a runtime architecture layered above existing language models.

The architecture does not require modification of the underlying neural network or transformer implementation.

Instead, RSCP interacts with the model through the standard inference interface, Because RSCP operates as a runtime layer above the inference engine, the architecture remains compatible with emerging model execution techniques including:

• speculative decoding
• prefix-resident inference scheduling
• continuous batching
• GPU key-value cache residency
• hardware accelerator scheduling

These techniques may improve throughput while the RSCP runtime continues to enforce deterministic reasoning memory constraints and verification guarantees.

This design allows RSCP to operate with:

• open-source models
• proprietary models
• cloud inference services
• embedded inference runtimes

As a result, RSCP can be integrated into existing AI deployments without requiring fundamental changes to model architectures.

6.6 Infrastructure Implications

The RSCP runtime architecture introduces several infrastructure benefits:

Predictable memory usage

Bounded reasoning memory prevents uncontrolled context growth.

Improved agent concurrency

More agents can operate within a fixed hardware environment.

Stable inference latency

Attention computation remains consistent over time.

Flexible knowledge scaling

External knowledge repositories can grow independently of reasoning memory.

These properties make RSCP particularly suitable for large-scale agent deployments and edge computing environments where hardware resources must be carefully managed.

Prefix-Resident Inference Scheduling

RSCP exposes shared reasoning prefixes explicitly through prefix-plus-delta execution. This creates a natural interface for future GPU-aware runtimes to maintain hot prefixes as resident inference state and route incoming tasks according to prefix locality. Such an approach may reduce repeated KV-state construction and improve agent density on accelerator hardware.

6.7 Prefix-Resident Scheduling (RIFT)

To explore how RSCP could interact with GPU inference runtimes, we implemented a simulation layer called **RIFT (Resident Inference Frame Table)**.

RIFT models a system where frequently reused reasoning prefixes remain resident within a limited number of GPU memory slots. When a task executes, the runtime checks whether the required reasoning prefix is already resident. If the prefix is resident, only the task-specific delta frame must be processed.

This simulates reuse of the prefix KV state in modern transformer inference runtimes.

Experimental configuration

The benchmark used:

- **100 concurrent agent tasks**
- **3 simulated prefix residency slots**
- RSCP runtime scheduling with prefix pinning and affinity grouping
- Focus Frame bounded reasoning context
- Oracle verification and Governor enforcement

Results

Metric	Result
Verification success	**100 / 100 tasks**
Prefix reuse rate	**94%**
Prefix resident hit rate	**81%**
Prefix cache size	**6 prefixes**
Resident slots used	**3 / 3**
Estimated density uplift	**~2.0×**

No prefix evictions occurred during the benchmark, indicating that the runtime converged toward a stable set of domain reasoning prefixes.

Interpretation

The results show that RSCP tasks naturally converge toward a small set of reusable reasoning prefixes corresponding to domain knowledge structures.

When these prefixes remain resident in GPU memory, subsequent tasks can execute using only the delta frame. This reduces repeated context construction and lowers the per-agent incremental reasoning footprint.

Under the tested workload, this approach produced an estimated **~2× agent density improvement** relative to naive prompt-based execution.

Reliability

Importantly, all tasks passed the Oracle verification stage, demonstrating that scheduling optimizations did not affect deterministic output validation.

Results

The benchmark produced the following results:

Metric	Result
Verification success	**100 / 100 tasks**
Prefix reuse rate	**94%**
Prefix resident hit rate	**81%**
Resident prefix slots	**3 / 3 utilized**
Average estimated density uplift	**~2.01×**
Runtime duration	~135 seconds

No prefix evictions occurred during the benchmark, indicating the runtime converged on a stable set of domain prefixes.

Interpretation

These results suggest that RSCP naturally converges toward a small set of reusable reasoning prefixes representing domain knowledge structures (for example claims processing, legal support, and audit review).

When these prefixes remain resident in GPU memory, the runtime can process subsequent tasks using only the delta frame. This significantly reduces duplicated reasoning context and enables higher concurrency under fixed memory budgets.

The simulation indicates that prefix-resident scheduling may enable approximately **2× higher agent density** compared with naive prompt-based execution.

Importantly, verification accuracy remained **100%**, demonstrating that scheduling optimizations do not compromise deterministic output validation.

Runtime Locality Observation

Experimental results suggest that RSCP workloads naturally converge toward a small working set of reusable reasoning prefixes. In the evaluated prototype runtime, throughput was maximized when worker concurrency approximately matched the number of distinct resident reasoning prefixes. This behaviour indicates that RSCP not only bounds reasoning memory, but also exposes a form of **reasoning locality** that may be useful to accelerator runtimes. By aligning task scheduling with resident reasoning prefixes, future inference

systems may reduce repeated context reconstruction, improve cache locality, and increase effective accelerator utilization.

Future Runtime Optimizations

The current RSCP prototype executes tasks through a simplified worker pool. Future runtime implementations may introduce prefix-aware scheduling and reasoning program residency management. By grouping tasks that share reasoning prefixes, the runtime may improve cache locality, reduce repeated context reconstruction, and increase effective accelerator utilization. Such scheduling strategies could allow RSCP runtimes to maintain resident reasoning programs while distributing task deltas across worker pools.

Runtime experiments indicate that optimal throughput occurs when worker concurrency approximately matches the number of resident reasoning prefixes. When worker counts exceed the active prefix working set, scheduling overhead and memory contention may reduce efficiency

7. Edge and Autonomous Systems

While large-scale datacenter deployments dominate current AI infrastructure discussions, many emerging applications require reliable AI systems operating under significantly tighter resource constraints.

These environments include:

• mobile devices
• robotics platforms
• autonomous vehicles
• industrial control systems
• edge computing infrastructure

In these settings, AI systems must operate within bounded computational resources while maintaining reliable and predictable behaviour.

Traditional large-context reasoning approaches are poorly suited for such deployments due to their reliance on expanding prompt context and unpredictable inference behaviour.

The RSCP runtime architecture addresses these challenges by introducing bounded reasoning memory and deterministic verification mechanisms.

7.1 Resource Constraints in Edge Systems

Edge AI systems often operate under strict hardware limitations.

Examples include:

Platform	Typical Constraints
Mobile devices	limited memory and battery
Robotics systems	real-time latency requirements
Embedded controllers	restricted compute resources
Autonomous vehicles	safety-critical reliability

In these environments, uncontrolled context expansion can quickly exhaust available memory or introduce unacceptable latency.

By enforcing a bounded reasoning envelope, RSCP ensures that reasoning memory usage remains predictable regardless of task duration.

This property allows AI agents to operate within resource-constrained environments without risking runaway memory consumption.

7.2 Deterministic Verification for Safety

Edge AI deployments frequently involve tasks where incorrect outputs can produce real-world consequences.

Examples include:

• financial transaction systems
• industrial automation
• autonomous navigation
• medical decision support

In such environments, purely probabilistic outputs from language models may not be sufficient.

The RSCP Oracle introduces a deterministic verification layer that checks generated outputs against trusted evidence stored within the Evidence Vault (l2 Knowledge layer).

This architecture enables systems to:

• detect incorrect outputs
• enforce policy compliance
• correct inconsistent data
• prevent propagation of hallucinated results

By integrating verification into the runtime pipeline, RSCP provides a mechanism for improving reliability in safety-sensitive AI systems.

7.3 Autonomous Agent Systems

Autonomous agents often operate over extended time horizons and interact with multiple external tools or data sources.

These systems typically require:

• persistent task memory
• knowledge retrieval
• iterative reasoning
• error correction

Traditional prompt-based reasoning approaches can become unstable during long-running operations as context windows expand and reasoning state becomes increasingly difficult to manage.

RSCP addresses this challenge by separating reasoning state from long-term knowledge storage.

Knowledge is maintained within the Evidence Vault (l2 Knowledge layer), while the Focus Frame maintains a bounded workspace for active reasoning.

This architecture allows agents to operate for extended periods without accumulating excessive context.

7.4 Local AI and Offline Operation

Another emerging trend in AI deployment is the shift toward local or on-device inference.

Advances in efficient models and specialized hardware have made it possible to run capable language models directly on consumer devices.

However, local inference systems must operate without the large computational resources available in cloud datacenters.

RSCP is well suited to such environments because the architecture:

• bounds reasoning memory
• reduces unnecessary context processing
• enables deterministic verification without cloud access

These properties allow RSCP-based systems to operate effectively in environments where connectivity may be limited or intermittent.

7.5 Hybrid Cloud–Edge Deployments

Many real-world systems combine cloud infrastructure with edge-based reasoning.

In such architectures:

• the cloud may host large knowledge repositories
• edge devices perform local reasoning tasks
• agents operate across distributed environments

The RSCP architecture naturally supports this model because the Evidence Vault (l2 Knowledge layer) can be distributed across multiple storage locations while the Focus Frame remains local to the reasoning runtime.

This separation allows knowledge repositories to scale independently of the reasoning environment.

7.6 Deployment Implications

By combining bounded reasoning memory with deterministic verification, RSCP enables several deployment advantages for edge and autonomous systems:

Predictable resource usage

Bounded reasoning prevents uncontrolled memory growth.

Improved reliability

Verification mechanisms reduce hallucination risk.

Scalable knowledge access

External knowledge repositories allow large information stores without expanding reasoning context.

Flexible deployment models

RSCP can operate in cloud, edge, or hybrid environments.

These properties make RSCP particularly suitable for emerging AI deployments where reliability, efficiency, and resource predictability are essential.

---

8. Related Work

Several approaches have been proposed to improve the efficiency and reliability of large language model inference systems.

Retrieval-Augmented Generation (RAG) systems introduce external document retrieval to supplement model knowledge. While these approaches improve factual grounding, retrieved content is typically appended directly to the prompt context, which can increase the size of the reasoning window.

Context window expansion techniques focus on increasing the maximum token capacity of transformer models through architectural changes or memory compression methods. While effective for certain workloads, these approaches still rely on processing large attention contexts.

Agent memory systems provide mechanisms for storing information across multiple interactions. However, many existing systems allow reasoning context to grow over time, potentially leading to increased memory consumption and reduced inference efficiency.

RSCP differs from these approaches by introducing a runtime architecture that **bounds reasoning memory independently of total knowledge size**, while optionally enforcing deterministic verification of model outputs. This design allows reasoning memory to remain stable while knowledge repositories scale externally.

8.a Runtime Observations

Short section:

The runtime benchmarks revealed two notable properties of the RSCP architecture:

Stable inference latency
Average latency remained approximately constant across workloads ranging from 50 to 500 tasks.

High reasoning prefix reuse
Tasks sharing the same reasoning program reused a single prefix across 751 requests.

These results suggest that many structured agent workloads can be represented as reusable reasoning programs with parameterized task inputs.

8.b Runtime Discovery

During multi-agent runtime experiments we observed that many structured workloads converge toward a small working set of reusable reasoning programs.

When these reasoning prefixes remain resident in memory, repeated prompt reconstruction can be avoided, allowing agent runtimes to significantly improve throughput and reasoning efficiency.

This observation suggests that future AI runtimes may benefit from treating reasoning prefixes as **persistent executable reasoning programs rather than transient prompt fragments**.

---

9. Conclusion

The rapid growth of agent-based AI systems has exposed fundamental limitations in traditional prompt-based reasoning architectures. As context windows expand, memory consumption increases, inference latency grows, and system reliability becomes harder to control.

This paper introduced **RSCP (Reasoning State Control Plane)**, a runtime architecture designed to address these limitations by separating reasoning state from knowledge storage and enforcing bounded reasoning memory.

RSCP introduces a structured execution pipeline composed of:

- a bounded reasoning envelope
- an external evidence repository
- an admission control layer
- optional deterministic verification mechanisms

Together these components convert reasoning state from an uncontrolled prompt history into a **managed runtime resource**.

Experimental evaluation demonstrates that RSCP can maintain deterministic correctness across benchmark workloads while operating within a bounded reasoning environment. Across multi-agent benchmark runs exceeding one thousand tasks, the system maintained stable latency and achieved a **100% verified task completion rate**.

Runtime experiments also suggest that structured workloads converge toward a **small working set of reusable reasoning programs**. Maintaining these programs as resident prefix states may allow accelerator runtimes to significantly reduce repeated context reconstruction and improve multi-agent throughput.

Observed performance characteristics indicate that optimal throughput occurs when worker concurrency approximately matches the number of resident reasoning prefixes.

The current prototype demonstrates approximately **2× logical density uplift** and approximately **3.5× useful-work improvement** under the evaluated workload. In deployments with stronger prefix locality, continuous batching, and GPU-native KV residency, the architecture may plausibly support substantially larger multi-agent density gains.

More broadly, RSCP suggests a shift in how AI systems manage reasoning memory. Rather than treating prompt context as an expanding record of interaction history, RSCP treats reasoning memory as a **bounded runtime resource similar to process memory in operating systems**.

The architecture is compatible with existing language models and can be deployed across cloud infrastructure, edge platforms, and embedded AI systems.

As AI systems evolve toward large populations of autonomous agents operating in complex environments, runtime architectures such as RSCP may provide a foundational layer for scalable, reliable, and resource-efficient AI reasoning systems.

Knowledge Scaling Behaviour

Experimental observations also indicate that the RSCP architecture decouples reasoning cost from total knowledge repository size. As the Evidence Vault grows, the Librarian admission controller selects only a small set of task-relevant evidence cards for the Focus Frame. This allows reasoning memory to remain bounded even as the external knowledge repository expands. As a result, inference latency remains stable while the knowledge base scales independently. This property enables RSCP systems to support continuously growing knowledge repositories without increasing the computational cost of individual reasoning tasks.

Runtime architectures that treat reasoning memory as a bounded and schedulable system resource may form a foundational layer for scalable multi-agent AI infrastructure.

Beyond immediate efficiency gains, RSCP suggests a path toward industry-standard reasoning control planes that coordinate memory residency, evidence admission, and verification independently of underlying model architectures.

10. Failure Modes and System Limitations

While the RSCP architecture introduces bounded reasoning memory and deterministic verification mechanisms, several limitations and potential failure modes must be considered.

10.1 Evidence Quality

The RSCP verification model assumes that trusted evidence stored within the Evidence Vault (l2 Knowledge layer) is accurate and authoritative.
If incorrect or outdated evidence is admitted into the vault, the Oracle may validate outputs that reflect incorrect underlying data.

Mitigation strategies include:

• evidence provenance tracking
• multi-source verification
• human review workflows
• capsule version control

These mechanisms ensure that the Evidence Vault (l2 Knowledge layer) remains a reliable knowledge source.

10.2 Retrieval Errors

The Librarian admission controller determines which evidence artifacts are admitted into the Focus Frame.
If relevant evidence is not retrieved, the language model may generate outputs based on incomplete information.

Possible mitigations include:

• improved semantic retrieval mechanisms
• namespace constraints
• capsule priority weighting
• admission policy tuning

Because RSCP maintains a bounded reasoning envelope, retrieval quality becomes an important factor in overall system performance.

10.3 Model Reasoning Limitations

RSCP does not modify the internal reasoning capabilities of the underlying language model.
If a task requires reasoning beyond the capabilities of the model, the system may still produce incorrect intermediate outputs.

However, the Oracle verification mechanism can detect many classes of errors and trigger regeneration cycles.

Future work may integrate specialized reasoning modules or domain-specific models to further improve performance.

10.4 Adversarial Inputs

Malicious or adversarial inputs may attempt to manipulate the reasoning process by injecting misleading instructions or data.

RSCP partially mitigates this risk through:

• bounded reasoning envelopes
• namespace-based evidence admission
• deterministic verification rules

Nevertheless, adversarial prompt attacks remain an active research area in AI safety.

10.5 Runtime Overhead

The RSCP runtime introduces additional coordination layers including admission control, verification, and lifecycle management.
Although these layers provide improved reliability, they may introduce small computational overhead compared to direct model inference.

However, experimental results suggest that this overhead is minor relative to the cost of large-context transformer inference.

10.6 Future Work

Future extensions of the RSCP architecture may include:

• distributed Evidence Vault (l2 Knowledge layer) across multiple nodes
• GPU-aware runtime schedulers for large-scale agent workloads
• advanced capsule merging and lifecycle policies
• multi-agent coordination frameworks
• hardware-assisted reasoning memory controllers

These developments could further improve the scalability and reliability of RSCP-based AI systems.

11. Future Runtime Optimizations

While the current RSCP prototype demonstrates deterministic reasoning execution using a bounded reasoning envelope and admission-controlled knowledge retrieval, the architecture is designed to integrate with a range of emerging inference optimizations and accelerator scheduling techniques.

Because RSCP operates as a runtime layer above the inference engine, these optimizations can be incorporated without modifying the underlying reasoning memory model. The RSCP runtime therefore provides a stable architectural foundation on which future inference performance improvements may be deployed.

The following subsections describe several optimization approaches that naturally align with the RSCP execution model.

11.1 Prefix-Resident Inference Scheduling

Many reasoning tasks share similar structural patterns.
For example, multiple agents performing claim validation, compliance checks, or audit analysis may use nearly identical reasoning instructions while varying only the task parameters.

RSCP exposes this structure explicitly through the **prefix plus delta execution model**.

```
Shared Reasoning Prefix
+
Delta Frame (Task Parameters)
```

Because these prefixes represent reusable reasoning programs, inference runtimes can maintain frequently used prefixes as resident state within accelerator memory.

Under this model, incoming tasks referencing the same reasoning prefix can reuse the existing prefix state rather than reconstructing the full reasoning context.

This approach can reduce repeated context processing and improve runtime efficiency for multi-agent workloads.

The experimental RIFT simulation presented earlier demonstrates that many structured workloads converge toward a small working set of reusable reasoning prefixes. Maintaining these prefixes in accelerator memory may significantly reduce repeated inference overhead.

11.2 GPU Key-Value Cache Residency

Modern transformer inference systems rely heavily on key-value attention caches to store intermediate attention state across token generation.

In conventional prompt-based systems, the KV cache must be rebuilt whenever the prompt context changes significantly. Large prompt contexts therefore lead to repeated KV cache reconstruction and increased memory traffic.

RSCP reduces this overhead by maintaining a bounded reasoning envelope and by separating reusable reasoning prefixes from task-specific parameters.

When prefixes remain stable across tasks, inference runtimes may retain corresponding KV states within accelerator memory, allowing subsequent tasks to reuse these states directly.

This behaviour can significantly reduce memory bandwidth requirements and may improve inference throughput for workloads with high prefix reuse.

Importantly, RSCP maintains the deterministic residency invariant of the reasoning envelope regardless of how KV cache optimizations are implemented within the inference runtime.

The RSCP prefix-plus-delta execution model exposes reusable reasoning programs that may remain resident within accelerator memory, enabling scheduling optimizations similar to instruction caching in traditional computing systems.

11.3 Speculative Decoding Integration

Speculative decoding techniques use a smaller draft model to generate candidate tokens that are then verified by a larger target model.

These techniques can significantly increase token generation throughput by allowing multiple tokens to be processed per inference pass.

RSCP is compatible with speculative decoding architectures because the runtime operates independently of the token generation strategy used by the inference engine.

The bounded Focus Frame ensures that speculative decoding operates on a stable reasoning context, while the Oracle verification layer provides an additional correctness check after generation.

This combination allows speculative decoding to improve inference speed while preserving deterministic output validation.

11.4 Accelerator Affinity Scheduling

As AI infrastructure evolves toward large multi-GPU and heterogeneous accelerator clusters, efficient scheduling of reasoning tasks becomes increasingly important.

The RSCP runtime exposes several properties that enable improved scheduling strategies:

• bounded reasoning memory per agent
• reusable reasoning prefixes
• structured evidence retrieval patterns
• explicit reasoning program identifiers

These properties allow runtime schedulers to route tasks toward accelerators that already host the required reasoning prefixes or cached inference state.

Such **affinity scheduling** may reduce memory transfers and improve throughput in large-scale agent deployments.

11.5 Continuous Batching and Multi-Agent Execution

Many inference systems use continuous batching techniques to combine multiple token generation requests into shared inference passes.

RSCP supports this model naturally because each reasoning session operates within an independent bounded Focus Frame.

Multiple reasoning sessions may therefore be scheduled concurrently while sharing:

• model weights
• prefix state
• accelerator execution pipelines

The deterministic memory footprint of the Focus Frame allows infrastructure operators to predict how many concurrent reasoning sessions can operate on a device without risking memory exhaustion.

This property is particularly valuable in multi-agent systems where large numbers of concurrent reasoning tasks may be active.

11.6 Hardware Co-Design Opportunities

The separation between reasoning memory and external knowledge storage introduced by RSCP opens potential opportunities for future hardware co-design.

Accelerator vendors may implement dedicated hardware support for:

• bounded reasoning memory management
• prefix residency tables
• evidence retrieval acceleration
• verification logic integration

Such hardware support could further improve the efficiency of large-scale AI reasoning systems while preserving the RSCP runtime architecture.

Importantly, the RSCP design remains compatible with existing inference hardware and does not require specialized accelerators.

11.7 Architectural Implications

The optimizations described above illustrate that RSCP is not merely a prompt management strategy but a runtime architecture capable of coordinating reasoning execution across evolving inference systems.

Because RSCP separates reasoning state, knowledge storage, and verification into independent subsystems, improvements in model architectures or accelerator hardware can be incorporated without changing the fundamental reasoning memory model.

This separation allows RSCP to remain compatible with future developments in AI inference infrastructure while continuing to enforce deterministic reasoning memory and structured knowledge governance.

12. Economic Implications of Bounded Reasoning Memory

Modern large language model deployments face significant infrastructure costs due to the rapid growth of prompt context and memory consumption during inference. As conversational sessions and agent workflows grow longer, the amount of contextual information included in each prompt expands, increasing both memory bandwidth usage and accelerator occupancy.

This growth produces several operational challenges:

• increased GPU memory pressure
• reduced inference throughput
• higher per-request energy consumption
• limited concurrency for agent workloads

RSCP addresses these issues by introducing a deterministic upper bound on active reasoning memory. By separating persistent knowledge storage from the bounded reasoning workspace used during inference, RSCP prevents prompt context from expanding indefinitely as session history grows.

This architectural constraint enables more predictable resource allocation and improves overall system efficiency.

12.1 Improved Agent Density

One of the most significant benefits of bounded reasoning memory is increased **agent density per accelerator**.

Traditional prompt-based systems scale poorly when many agents operate concurrently because each agent must maintain a large prompt context representing accumulated interaction history.

Under RSCP, the reasoning memory used by each agent is limited to the size of the Focus Frame.

Historical interaction data is stored externally in the Evidence Vault and retrieved selectively through the admission controller.

This design allows many concurrent reasoning sessions to share the same model weights and accelerator resources without duplicating large prompt contexts.

Experimental results from the prototype implementation demonstrate that bounded reasoning envelopes can significantly reduce active prompt size while maintaining deterministic output verification.

As a result, infrastructure operators may be able to run substantially more agents per accelerator compared with traditional prompt-based architectures.

12.2 Reduced Memory Bandwidth Pressure

Transformer inference workloads are frequently limited by memory bandwidth rather than raw compute capability.

Each generated token requires repeated access to model weights and attention state, making memory movement a dominant factor in inference performance.

Large prompt contexts amplify this problem because attention operations must process every token in the prompt window.

RSCP mitigates this effect by reducing the amount of contextual information present in the reasoning workspace at any given time.

Because the Focus Frame contains only the subset of evidence required for the current reasoning step, attention computation operates over a smaller token set.

This reduction can lower memory bandwidth pressure and improve effective inference throughput.

12.3 Lower Operational Costs for AI Infrastructure

The economic impact of bounded reasoning memory becomes particularly significant at large scale.

In hyperscale deployments, even small improvements in inference efficiency can translate into substantial cost reductions across large clusters of accelerator hardware.

Potential benefits include:

• increased agent throughput per GPU
• reduced memory bandwidth usage
• lower energy consumption per request
• improved utilization of accelerator clusters

These factors collectively reduce the cost of operating large-scale AI systems.

Because RSCP functions as a runtime layer above existing model architectures, these benefits may be realized without requiring new model training or specialized hardware.

12.4 Enabling Edge AI Deployment

Edge devices such as mobile phones, embedded systems, and robotics platforms face severe constraints on memory capacity and energy consumption.

Traditional prompt-based reasoning architectures are difficult to deploy in these environments because prompt context can grow beyond the available memory resources of the device.

RSCP's bounded reasoning architecture is particularly well suited to edge deployment.

Because the Focus Frame maintains a deterministic memory footprint, inference workloads can be sized to fit within the constraints of edge hardware.

External knowledge repositories may reside locally on device storage or within nearby edge nodes, while the runtime selectively retrieves only the information required for the current reasoning step.

This architecture enables reliable AI reasoning on devices that would otherwise be unable to support large prompt contexts.

12.5 Reliability and Governance Benefits

In addition to performance improvements, RSCP introduces mechanisms that improve reliability and governance of AI reasoning systems.

The Oracle verification layer allows generated outputs to be compared with known evidence artifacts or witness records. Discrepancies can be flagged or corrected before results are returned to the user.

The Governor mechanism detects repeated reasoning failures or non-progressing execution patterns and may halt execution to prevent uncontrolled agent behavior.

These features are particularly valuable for high-risk domains such as:

• medical decision support
• financial compliance systems
• legal document analysis
• autonomous robotic systems

By combining bounded reasoning memory with deterministic verification and governance mechanisms, RSCP provides a foundation for more trustworthy AI systems.

12.6 Implications for AI Runtime Operating Systems

Taken together, the architectural and economic benefits of RSCP suggest a shift in how AI systems manage reasoning execution.

Rather than treating prompt context as an ever-expanding interaction history, RSCP treats reasoning memory as a managed runtime resource analogous to process memory in traditional operating systems.

This perspective allows reasoning workloads to be scheduled, bounded, and verified in ways that more closely resemble established operating system design principles.

As AI systems evolve toward large multi-agent deployments, runtime architectures that manage reasoning memory deterministically may become essential infrastructure for scalable AI platforms.

Diagram 1 — L2 Evidence Lifecycle

Use this in both the white paper and the patent.

Figure: L2 Card and Capsule Lifecycle

```
                +----------------------+
                |   Raw Evidence Card  |
                | tool output / input  |
                +----------+-----------+
                           |
                           v
                +----------------------+
                | Verification Engine  |
                | Oracle / witness     |
                +----------+-----------+
                           |
                 +---------+---------+
                 |                   |
                 v                   v
       +-------------------+   +-------------------+
       | Verified Fact Card |   | Rejected / Flagged|
       +---------+---------+   +-------------------+
                 |
                 v
       +------------------------+
       | Topic Capsule Builder  |
       | merge / deduplicate    |
       +-----------+------------+
                   |
                   v
       +------------------------+
       | Active Capsule (Hot L2)|
       | high-priority knowledge|
       +-----------+------------+
                   |
           +-------+-------+
           |               |
           v               v
+----------------+   +----------------+
| Warm Storage   |   | Cold Archive   |
| aging evidence |   | stale evidence |
+----------------+   +----------------+
```

Caption

Figure 1. Lifecycle of evidence within the RSCP Evidence Vault (l2 Knowledge layer). Raw evidence is verified before being promoted into fact cards, merged into topic capsules, maintained in active storage, and progressively aged into warm or cold storage.

Diagram 2 — L1 ↔ L2 Movement and Focus Stability

This one is very important.

Figure: Focus Frame Admission and Memory Flow

```
                    +----------------------+
                    |   Agent Task / Goal  |
                    +----------+-----------+
                               |
                               v
                    +----------------------+
                    |     Focus Frame      |
                    | bounded L1 workspace |
                    +----------+-----------+
                               |
                               v
                    +----------------------+
                    |      Librarian       |
                    | admission control    |
                    +----------+-----------+
                               |
            +------------------+------------------+
            |                                     |
            v                                     v
+------------------------+             +------------------------+
| Active Topic Capsule   |             | Supporting Capsules    |
| red / orange priority  |             | yellow / blue priority |
+-----------+------------+             +-----------+------------+
            |                                       |
            +------------------+-------------------+
                               |
                               v
                    +----------------------+
                    | Evidence Cards to L1 |
                    | admitted under quota |
                    +----------+-----------+
                               |
                               v
                    +----------------------+
                    |   LLM Draft Output   |
                    +----------+-----------+
                               |
                               v
                    +----------------------+
                    | Oracle Verification  |
                    +----------+-----------+
                               |
                 +-------------+-------------+
                 |                           |
                 v                           v
     +------------------------+   +------------------------+
     | Verified Result        |   | Correction / New Card  |
     | emit output            |   | promote back to L2     |
     +------------------------+   +------------------------+
```

Caption

Figure 2. Movement of knowledge between the bounded L1 Focus Frame and the external L2 Evidence Vault. The Librarian admits cards into L1 under bounded quotas, while verified outputs and corrections are promoted back into L2.

Diagram 3 — RSCP Layer Map

This should go into the white paper exactly as the stack definition.

Figure 1: RSCP Layer Map

```
+---------------------------------------------------+
| Application Layer                                 |
| User UI / API / External Intent                   |
+---------------------------------------------------+
| Agent Layer                                       |
| Intent Planner / Task Decomposition               |
+---------------------------------------------------+
| RSCP Runtime (Control Plane)                      |
| Focus Frame / Governor / Oracle / Quotas          |
+---------------------------------------------------+
| Evidence Layer (Knowledge Plane)                  |
| Librarian / Cards / Capsules / Vault Lifecycle    |
+---------------------------------------------------+
| Inference Layer (Execution Plane)                 |
| Model / KV Cache / Decode / Attention             |
+---------------------------------------------------+
| Hardware Layer (Physical Plane)                   |
| GPU / TPU / NPU / CPU / Memory Bandwidth          |
+---------------------------------------------------+
```

Caption

Figure 3. Layered architecture of the RSCP Runtime OS showing the separation between application intent, agent planning, runtime control, evidence management, inference execution, and hardware.

Figure 2 — RSCP Reasoning Memory Hierarchy

```
Applications
      │
Agent Planner
      │
RSCP Runtime
 ├ Focus Frame
 ├ Librarian
 ├ Oracle
 └ Governor
      │
Prefix Cache (L1)
Reusable reasoning programs
      │
Evidence Vault (L2)
Fact cards and capsules
      │
Inference Engine
```

This visualizes the architecture very clearly.

Diagram 4 — RSCP Runtime OS

```
        +---------------------------+
        |       Applications        |
        |  Agents / Tools / APIs    |
        +------------+--------------+
                     |
                     v
        +---------------------------+
        |      Intent Planner       |
        |  Task Decomposition       |
        +------------+--------------+
                     |
                     v
+-------------------------------------+
|        RSCP Runtime Kernel          |
|                                     |
|  Focus Frame Manager                |
|  Librarian Admission Controller     |
|  Oracle Verification Engine         |
|  Governor Stability Monitor         |
+-------------------+-----------------+
                     |
                     v
        +---------------------------+
        |        Evidence Vault     |
        |   Cards / Capsules / L2   |
        +------------+--------------+
                     |
                     v
        +---------------------------+
        |      Inference Engine     |
        |  LLM / Neural Model       |
        +------------+--------------+
                     |
                     v
        +---------------------------+
        |       Hardware Layer      |
        | GPU / TPU / CPU / NPU     |
        +---------------------------+
```

Caption:

RSCP operates as a runtime kernel coordinating reasoning memory, knowledge access, verification, and execution stability for AI workloads.

Diagram 5 - RSCP Knowledge Tier Architecture

```
   L3 Distributed Knowledge
cloud / edge / external storage
           ▲
           |
   +------------------+
   |   Evidence Vault |
   |        L2        |
   +---------+--------+
             |
             ▼
   +------------------+
   |   Focus Frame    |
   |       L1         |
   | bounded reasoning|
   +------------------+
```

References

References

1. Vaswani, A., et al. (2017). *Attention Is All You Need.* NeurIPS.
2. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS.
3. Song, J., et al. (2023). *Speculative Decoding for Large Language Model Inference.*
4. Yang, D., et al. (2025). *Learned Prefix Caching for Efficient LLM Inference.*
5. Staniszewski, K., et al. (2025). *KV Cache Transform Coding for Compact Storage in LLM Inference.*
6. Pan, R., et al. (2025). *Marconi: Prefix Caching for Hybrid LLM Systems.*
7. Gill, W., et al. (2025). *Semantic Caching for Large Language Models.*
8. Kwon, W., et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM).*

Correspondence

The author welcomes discussion with researchers, engineers, and organizations interested in exploring the RSCP architecture.

Graeme Randle
Independent Developer Rustic Labs
Brisbane, Australia
Contact: gsemailaddress@gmail.com